NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 12: Searching and Sorting Algorithms

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Specify the result. Search deliberately.
Preserve records. Verify the evidence.

SCHOOL OF ENGINEERING

About these notes

These notes accompany the **90-page Week 12 presentation, Searching and Sorting Algorithms**, previously shared for ENG 101. They expand the same three lectures into a student reading guide. The examples, contracts, datasets, activity questions, and numerical count records in that presentation determine the sequence. This document does not substitute a different collection of searching or sorting algorithms.

Course information

Course: ENG 101 — Programming for Engineers; lecture section 2L; 6 ECTS.
Instructor: Dr. Yanwei Wang, School of Engineering, Nazarbayev University.
Term: Fall 2026.
Week 12: Search algorithms, brute-force search, linear search, bisection search, and sorting algorithms. The source scope statement identifies CLOs 1 and 2 and preparation for Assignment 7.

Lecture-slide alignment: Week 12 source PDF, pp. 1–3.

A reading route through the week

Lecture	Question to keep asking
34: Linear and brute-force search	What must be returned: a Boolean, a first index, all matches, or one best feasible record?
35: Bisection search in ordered data	Why is it valid to discard this part of the candidate interval?
36: Sorting algorithms and engineering rankings	Which order is required, and which relationships and repeated values must remain intact?

Functions, lists, dictionaries, recursion, tests, and efficiency analysis from earlier weeks are tools for this work. You are not being asked to memorize an isolated recipe. You are learning to choose an algorithm whose result has the meaning needed by the engineering task.

Key idea

Specify the result → search deliberately → reorder with meaning → verify the evidence. A faster answer is useful only when it answers the right question. In particular, sorting a time series changes the meaning of its positions, and finding an insertion position does not prove that a target was stored.

How to study a code example

Read its contract first. Identify the input order, units, equality or tolerance rule, absence marker, duplicate policy, and promised mutation behavior. Predict a tiny case before executing it. Trace the variables that control progress, then explain why the returned result satisfies the contract. Finally, test an empty input, a boundary, duplicates where relevant, and an absent or infeasible result.

All Python listings contain comments. Numbered files in `examples/` are runnable demonstrations; keep `week12_algorithms.py` and `week12_counts.py` beside them. Listings labelled as *definitions* are components, not stand-alone console programs. The complete companion modules supply their imports. The separate `ENG101_Week12_Annotated_Examples.py` is self-contained and runs a selected sequence of non-defective demonstrations without additional packages.

What comes from the slides, and what is added?

The earlier Week 12 algorithm and counting modules were recovered and retained unchanged. Some slides display only a definition, a diagram, or an excerpt from a driver. Complete client programs, added comments, tests, activity answers, and cumulative exercises make these examples usable for self-study. They are labelled as worked completions or extensions, not represented as verbatim slide content. Technical qualifications are also labelled where necessary.

Several numbered demonstrations intentionally expose an incorrect test, wrong update, violated precondition, or sorting defect. Their explanations identify what is wrong. The no-progress example is bounded to two passes so that studying it does not hang a program. Do not copy an intentional-defect example into a solution as though it were reference code.

Contracts are not automatically validation routines

Many elementary functions assume finite values with compatible units and meaningful comparisons. They do not validate every input. The calibration-grid helper does explicitly validate its data. These choices are kept distinct: adding an $O(n)$ validation pass inside an $O(\log n)$ lookup changes the cost of the full operation. Illustrative densities, stresses, catalogue values, costs, and feasibility limits are programming data, not certified engineering specifications.

Counts, measurements, and source numbering

The two source CSV files contain *selected-operation counts*, not elapsed times. The search plot counts linear equality tests and binary midpoint probes separately. The sorting plot contains comparison counts reported for seeded shuffled inputs; the available source package does not retain the seed or permutations. The original values are preserved, but exact regeneration of those merge counts from the absent permutations is not claimed. An additional reproducible experiment is clearly separated from them.

References such as “source page 38” mean **physical PDF page 38 of the 90-page deck**. Its footer slide numbers differ because divider pages are not counted in the same way. Appendix A.3 maps every source page to a section of these notes. Optional insertion sort, the manual lower-bound implementation, object sorting, and the supplementary comparison plot remain optional. Assignment 7 requirements remain those announced on Moodle.

Contents

About these notes 2

1. Linear and Brute-Force Search 6
1.1. One dataset can answer several different questions . . . 6
1.2. The linear-search algorithm and its invariant . . . 7
1.3. Absence is not the same as index zero . . . 8
1.4. Search for a condition and interpret the location . . . 9
1.5. Do not sort away the question . . . 10
1.6. All matches require the whole scan . . . 10
1.7. Search names without losing the rest of the record . . . 11
1.8. First close and nearest are different contracts . . . 12
1.9. Brute force over a finite catalogue . . . 13
1.10. Search combinations without repeating them . . . 14
1.11. A running best must keep searching . . . 15
1.12. Debugging: absence reported too early . . . 17
1.13. Lecture 34 exit ticket: worked answers . . . 17

2. Bisection Search in Ordered Data 18
2.1. Order supplies a reason to eliminate candidates . . . 18
2.2. Preconditions are part of the algorithm . . . 18
2.3. An inclusive candidate interval . . . 19
2.4. Trace a successful and an unsuccessful search . . . 20
2.5. Progress and the smallest boundary tests . . . 21
2.6. An invalid assumption can produce a quiet wrong answer . . . 22
2.7. Duplicates, insertion positions, and exact wrappers . . . 22
2.8. A calibration interval has its own endpoint policy . . . 24
2.9. Keep the original index with the sorted view . . . 27
2.10. Count what actually happens . . . 27
2.11. Include preparation and repeated queries . . . 29
2.12. Activity, debugging checklist, and exit ticket . . . 29

3. Sorting Algorithms and Engineering Rankings 30
3.1. Ordered numbers can still describe the wrong record . . . 30
3.2. Two Python interfaces: new list or changed receiver . . . 31
3.3. Keys make a ranking criterion explicit . . . 31
3.4. Selection sort fixes one final position per pass . . . 33
3.5. Selection-sort cost and its stability limitation . . . 34
3.6. Merge sort divides first and combines on return . . . 35
3.7. A stable merge must preserve unconsumed tails . . . 35
3.8. Derive work and peak storage separately . . . 37
3.9. Test the entire sorting contract . . . 37
3.10. Separate constraints from the ranking objective . . . 38
3.11. One best record does not require a full ranking . . . 40
3.12. A sorted-looking result may have lost information . . . 40
3.13. Assignment preparation and worked exit ticket . . . 41

4. Optional Extensions and Comparison Evidence 42
4.1. Insertion sort builds a sorted prefix . . . 42

4.2. A manual lower-bound search uses a different invariant . . . 43
4.3. Sort complete objects and inspect stability . . . 44
4.4. Original comparison counts are not clock timings . . . 45
4.5. Choose the method by the required result . . . 46

5. Cumulative Practice and Worked Solutions 47
5.1. P1: write the contract before the loop . . . 47
5.2. P2: first close versus nearest . . . 47
5.3. P3: justify a catalogue decision . . . 48
5.4. P4: count a complete combination search . . . 48
5.5. P5: trace unsuccessful exact search . . . 48
5.6. P6: duplicate policies and safe indexing . . . 49
5.7. P7: distinguish a grid point from an interval . . . 49
5.8. P8: repair a detached record . . . 49
5.9. P9: count and interpret selection-sort passes . . . 50
5.10. P10: reason about merging and storage . . . 50
5.11. P11: audit a recommendation under changing priorities . . . 50
5.12. P12: distinguish a reference from a copied record . . . 50

6. Quick Reference and Study Checklist 52
6.1. Five output contracts to distinguish . . . 52
6.2. Compact cost reference . . . 52
6.3. Glossary . . . 52
6.4. Readiness for language transfer and course review . . . 54

A. Source Basis and Complete Slide Coverage 55
A.1. Primary basis and source distinctions . . . 55
A.2. Reading references . . . 55
A.3. Page-by-page alignment . . . 56

Bibliography 58

1 Linear and Brute-Force Search

Lecture-slide alignment: Week 12 source PDF, pp. 4–29; Lecture 34.

Learning outcomes

By the end of this chapter, you should be able to distinguish search contracts, implement and trace first-match linear search, inspect numerical and structured records, enumerate finite combinations, select a best feasible record, and design tests that expose incorrect early returns.

1.1. One dataset can answer several different questions

Lecture-slide alignment: Week 12 source PDF, pp. 5–8.

The source pressure list is `[95, 108, 99, 115, 102]`, in kPa and in acquisition order. Asking whether 115 is present is not the same as asking where it first appears. Asking which readings exceed 100 kPa is different again. The first answer is `True`; the second is index 3; the third contains three index–value pairs.

```
from week12_algorithms import linear_search, all_above
# Readings retain their original acquisition order; the unit is kPa.
values = [95, 108, 99, 115, 102]
print('Present:', 115 in values)  # Membership is a Boolean result.
print('First index:', linear_search(values, 115))
print('Above 100:', all_above(values, 100))  # Keep indices and values.
```

Code 1.1: Worked completion of the opening three-question activity.

```
Present: True
First index: 3
Above 100: [(1, 108), (3, 115), (4, 102)]
```

Definition

A **search contract** states what information the caller will receive and what each possible result means. The same loop pattern can implement different contracts, but the function name, output, and stopping rule must agree with the requested task.

Question	Result when successful	Absence
Does a match exist?	A Boolean truth value	`False`
Where is the first match?	One integer index	`None`
Which records match?	All matching records or labelled values	Empty list
Which feasible candidate is best?	One selected record	`None`

A contract must also specify whether equality is exact, whether a tolerance is involved, which order is meaningful, what happens with duplicates, and whether the original input may change.

A program that returns the last match instead of the first can produce a valid index and still be wrong. A program that removes repeated matches can produce an attractive-looking report and still violate its contract.

1.2. The linear-search algorithm and its invariant

Lecture-slide alignment: Week 12 source PDF, pp. 8–10.

Linear search examines candidates in order. At each position, it tests the stated predicate: a yes-or-no condition. An exact search uses equality. A first-match search returns immediately on success. Only after the entire sequence has been examined is there evidence for an absent result.

```
# Source companion definition; consult the surrounding notes for its contract.
def linear_search(values, target):
    """Return the FIRST exact-match index, or None; preserve input order."""
    for index, value in enumerate(values):
        # Earlier positions have already failed the equality test.
        if value == target:
            return index
    # Absence is established only after every candidate has been checked.
    return None
```

Code 1.2: Source definition: return the first exact-match index.

The loop `enumerate(values)` supplies two local names: `index` is the current position and `value` is the object at that position. The equality comparison asks about the element, not the entire list. A successful `return index` ends the function, not just that loop pass. The final `return None` is outside the loop because its conclusion depends on every earlier comparison having failed.

Invariant

Before position i is tested, every earlier position $0, \ldots, i-1$ has failed the same equality test. Thus, when a match is found at i, no earlier match exists. If the loop finishes, every position has failed and absence has been established.

This invariant supplies a small correctness argument. It is initially true because there are no earlier positions before index zero. A failed test extends the checked prefix by one. A successful test justifies the first-match return. Termination follows because a finite list has finitely many positions and each non-returning pass advances to the next one.

For target 115 kPa, the source trace is:

Index	Value (kPa)	Equal to 115?	Tests so far	Action
0	95	False	1	Continue
1	108	False	2	Continue
2	99	False	3	Continue
3	115	True	4	Return 3

Index 4 is not inspected. Its value cannot change the answer to a *first*-match question after index 3 has succeeded.

1.3. Absence is not the same as index zero

Lecture-slide alignment: Week 12 source PDF, pp. 11–13.

Python treats integer zero as false in a Boolean context. This is a property of truth testing, not evidence that zero is an invalid index. For an index-or-`None` contract, `if index:` is therefore the wrong way to decide whether a result was found.

```
from week12_algorithms import linear_search
# The first element is a successful match, not an absence marker.
index = linear_search([95, 108, 99], 95)
print('Index:', index)
print('Explicit found test:', index is not None)
print('Incorrect truthiness test:', bool(index))
```

Code 1.3: A successful search whose result is zero.

```
Index: 0
Explicit found test: True
Incorrect truthiness test: False
```

The explicit test `index is not None` distinguishes absence from every valid integer position. There is no need to convert the index to a Boolean. Similarly, replacing `None` with -1 would require a different contract and careful handling because -1 is itself a valid Python index referring to the last element.

```
from week12_algorithms import linear_search
# Expected answers are supplied independently of the function body.
cases = [([], 7, None), ([7], 7, 0), ([7], 8, None),
         ([5, 7, 7], 7, 1), ([5, 7, 9], 9, 2)]
for values, target, expected in cases:
    before = values.copy()  # Additional check: searching must not reorder data.
    assert linear_search(values, target) == expected
    assert values == before
print('Five linear-search cases passed')
```

Code 1.4: Source regression cases with an added non-mutation check.

```
Five linear-search cases passed
```

These five cases cover an empty sequence, a present singleton, an absent singleton, repeated matches, and a match at the last position. The duplicate case is especially important: `[5, 7, 7]` with target 7 must return 1 under this function's first-match promise. Index 2 points to an equal value but is not an acceptable answer to this contract.

Stop, predict, and try it

Change the expected result for the duplicate test to 2 without changing the function. Predict which assertion fails and explain why changing the expected answer did not reveal a new implementation error.

Answer and explanation

The first-match implementation returns 1, so the altered equality assertion fails. The test expectation is now inconsistent with the contract. Tests are only useful when their expected results are justified independently and correctly.

1.4. Search for a condition and interpret the location

Lecture-slide alignment: Week 12 source PDF, pp. 14–15.

A useful engineering search often asks for a condition rather than a specific stored number. The source temperature task asks for the first reading *strictly above* 60.0 °C. A value exactly equal to 60.0 does not satisfy that predicate.

```
# Source companion definition; consult the surrounding notes for its contract.
def first_above(readings, limit):
    """Return the first index strictly above limit, or None; units must agree."""
    for index, reading in enumerate(readings):
        # Use the original measurement order, not a value-sorted copy.
        if reading > limit:
            return index
    return None
```

Code 1.5: Source definition: first strict threshold exceedance.

```
from week12_algorithms import first_above
# Finite readings in deg C; sampling begins at 0 s with a 2 s interval.
readings = [42.1, 45.3, 49.8, 53.2, 58.7, 62.4, 66.1]
index = first_above(readings, 60.0)
if index is not None:
    time_s = 2.0 * index  # Metadata convert a position into a time.
    print(f'Index {index}: {readings[index]:.1f} deg C at {time_s:.1f} s')
else:
    print('No reading above the stated limit')
```

Code 1.6: Locate a temperature event and use the stated sampling metadata.

```
Index 5: 62.4 deg C at 10.0 s
```

The first qualifying value is 62.4 °C at index 5. The index is dimensionless: it tells us which stored element was inspected. A time requires sampling information. For uniform sampling,

$$t_i = t_0 + i\Delta t.$$

The slide explicitly introduces $t_0 = 0$ s and $\Delta t = 2$ s, so the event is observed at 10 s. An index of 5 does not mean five seconds. If sampling intervals are unequal, the correct time must be retrieved from the timestamp associated with that reading; this uniform-spacing formula would no longer apply.

This is a first *recorded* threshold exceedance. The code does not locate the exact continuous time at which a physical temperature crossed 60 °C between samples. That would require an additional model or additional measurements.

1.5. Do not sort away the question

Lecture-slide alignment: Week 12 source PDF, p. 16.

Consider the acquisition sequence `[75, 95, 70, 85]`. The first reading above 80 is 95 at index 1. Sorting the values creates `[70, 75, 85, 95]`, in which the first value above 80 is 85 at index 2. Both computations can run correctly, but they answer different questions.

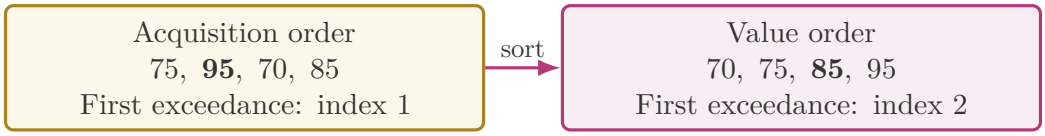


Figure 1.1.: The same values in a different order encode a different first-event question. Redrawn from source page 16.

```python
from week12_algorithms import first_above
# The same values describe two different orders.
raw = [75, 95, 70, 85]
ranked = sorted(raw)  # New value order, not a chronological record.
i = first_above(raw, 80)
j = first_above(ranked, 80)
print('Acquisition:', i, raw[i])
print('Value-sorted:', j, ranked[j])
print('Raw unchanged:', raw)
```

Code 1.7: Worked driver showing the change in question after sorting.

```
Acquisition: 1 95
Value-sorted: 2 85
Raw unchanged: [75, 95, 70, 85]
```

For chronology, spatial order, or an explicitly ranked catalogue, positions carry meaning. A value-sorted list is a useful derived view, but it is not a replacement for the original sequence. Keep raw data and state which coordinate system each returned index belongs to.

1.6. All matches require the whole scan

Lecture-slide alignment: Week 12 source PDF, p. 17.

If the caller needs *all* readings above a limit, the stopping rule must change. The function should append every match and continue to the end.

```python
# Source companion definition; consult the surrounding notes for its contract.
def all_above(readings, limit):
    """Return all (index, value) matches, retaining acquisition order."""
    matches = []
    for index, reading in enumerate(readings):
        if reading > limit:
            # Keep the measurement's location attached to its value.
            matches.append((index, reading))
    return matches
```

Code 1.8: Source definition: preserve every qualifying index–value pair.

For the opening pressure data and limit 100 kPa, the result is `[(1, 108), (3, 115), (4, 102)]`. A tuple keeps each location attached to its value. Appending matches in traversal order preserves acquisition order, and repeated qualifying readings remain separate observations.

Under the bounded-cost scalar model used in Weeks 10–11, the function performs n threshold comparisons for n readings. It stores k matches. For increasing n, time is $\Theta(n)$ and output storage is $\Theta(k)$; the fixed loop variables add constant auxiliary state. At an empty input there are no data comparisons, but initialization and return still occur. A first-match search can finish after one comparison; this all-match implementation cannot make that early exit while keeping its promise.

1.7. Search names without losing the rest of the record

Lecture-slide alignment: Week 12 source PDF, pp. 18–19.

A material record contains more than a name. Its density and unit must remain attached to the material that supplied them. The source lookup returns the first entire dictionary whose normalized name matches the normalized query.

```python
# Source companion definition; consult the surrounding notes for its contract.
def find_material(records, query):
    """First named material record or None; names are case-insensitive here."""
    # Normalize material names only; do not strip zeros from asset IDs.
    wanted = query.strip().lower()
    for record in records:
        if record["name"].strip().lower() == wanted:
            return record  # This is a reference, not an independent copy.
    return None
```

Code 1.9: Source definition: a case-insensitive material-name lookup.

```python
from week12_algorithms import find_material
# Illustrative densities; these values are not material specifications.
materials = [{'name': 'steel', 'density_kg_m3': 7850},
             {'name': 'aluminum', 'density_kg_m3': 2700}]
found = find_material(materials, ' Steel ')
if found is not None:
    print(found['name'], found['density_kg_m3'], 'kg/m^3')
print('Missing:', find_material(materials, 'copper'))
print('Same record object:', found is materials[0])  # Not a copy.
```

Code 1.10: Find the record, report its unit, and inspect identity.

```
steel 7850 kg/m^3
Missing: None
Same record object: True
```

Here `strip().lower()` is a stated naming policy: surrounding whitespace and capitalization do not distinguish material names. That does not authorize changing every engineering identifier. `P-007` and `P-7` may identify different assets. Cleaning must follow a data specification rather than a desire to make strings look simpler.

The returned dictionary is the same dictionary held in the input list. The search does not mutate it, but a later assignment through the returned reference would. A new outer list of records likewise does not imply independent record copies. For this flat record of immutable field values, `record.copy()` is sufficient when independent field replacement is intended.

A missing `"name"` field is different from an absent material. The former violates the record schema and can raise `KeyError`; the latter is a legitimate search result represented by `None`. Do not suppress a schema error and report that the material simply was not found.

1.8. First close and nearest are different contracts

Lecture-slide alignment: Week 12 source PDF, pp. 20–21.

For measured or approximate values, the predicate may be an absolute-tolerance test:

$$|v - v_{\text{target}}| \leq \varepsilon.$$

The tolerance has the same physical unit as the compared values. The source implementation assumes finite compatible numbers and $\varepsilon \geq 0$; it does not validate those conditions itself.

```python
# Source companion definition; consult the surrounding notes for its contract.
def first_close(values, target, tolerance):
    """First match within absolute tolerance, not necessarily the nearest.

    Assume finite numeric inputs in one unit and tolerance >= 0.
    """
    for index, value in enumerate(values):
        # Tolerance has the same physical unit as value and target.
        if abs(value - target) <= tolerance:
            return index
    return None
```

Code 1.11: Source definition: the first value satisfying an absolute tolerance.

```python
from week12_algorithms import first_close
# Voltages and tolerance are in the same unit, V.
values_v = [9.91, 10.02, 10.00]
index = first_close(values_v, 10.0, 0.1)
print('First close:', index)
# The exact match at index 2 has smaller error but was not requested.
print('Error of first:', f'{abs(values_v[index] - 10.0):.2f} V')
print('Error at index 2:', f'{abs(values_v[2] - 10.0):.2f} V')
```

Code 1.12: Worked answer to the first-close versus nearest activity.

```
First close: 0
Error of first: 0.09 V
Error at index 2: 0.00 V
```

The first result is index 0 because 9.91 V is within 0.1 V of 10.0 V. Index 2 has the smallest absolute error, but `first_close` never promised to find it. To find the nearest value in an unsorted list, retain the best error while inspecting all candidates, and state what happens on a tie. A worked extension appears in Chapter 5.

Engineering check

Exact matching remains appropriate for many integer catalogue values and identifiers. Tolerance is part of a particular numerical specification, not a universal repair for every equality test. Formatting a number to two decimal places changes its display, not the value used in the comparison.

1.9. Brute force over a finite catalogue

Lecture-slide alignment: Week 12 source PDF, pp. 22–23.

Brute-force search directly evaluates a stated set of candidates. It is not necessarily random, unreliable, or inefficient for the scale of the problem. A short catalogue of standard dimensions is often an excellent place to use a transparent exhaustive or sequential search.

The source area example assumes uniform axial stress $\sigma = F/A$, with $F = 12\,000$ N and allowable stress 80 MPa. The listed areas are 50, 75, 100, 125, and 150 mm^2, stored in square metres. The source helper returns the first listed feasible area.

```python
# Source companion definition; consult the surrounding notes for its contract.
def first_feasible_area(areas_m2, force_n, allowable_pa):
    """Return first acceptable listed area, or None, for uniform stress F/A.

    Assume finite positive areas/allowable and finite nonnegative force.
    The returned area is smallest only if the candidates are ascending.
    """
    for area_m2 in areas_m2:
        # N / m^2 = Pa. Equality at the stated limit is accepted.
        if force_n / area_m2 <= allowable_pa:
            return area_m2
    return None
```

Code 1.13: Source helper underlying the finite-area driver.

```python
from week12_algorithms import first_feasible_area
# Uniform stress F/A: force in N, areas in m^2, allowable stress in Pa.
areas_m2 = [50e-6, 75e-6, 100e-6, 125e-6, 150e-6]
area = first_feasible_area(areas_m2, 12000.0, 80e6)
if area is not None:
    print(f'Area = {area * 1e6:.0f} mm^2')  # Convert only for reporting.
    print(f'Stress = {12000.0 / area / 1e6:.1f} MPa')
else:
    print('No listed area meets the stated stress rule')
```

Code 1.14: The catalogue search with visible unit conversions.

```
Area = 150 mm^2
Stress = 80.0 MPa
```

An independent check is

$$A \geq \frac{12\,000}{80 \times 10^6} = 1.5 \times 10^{-4}\ \text{m}^2 = 150\ \text{mm}^2.$$

The stresses at the five candidate areas are 240, 160, 120, 96, and 80 MPa. Equality is accepted. Thus the fifth candidate succeeds exactly at the stated limit.

The first feasible area is also the smallest listed feasible area *because the catalogue is ascending.* With an arbitrary order, that optimality conclusion would not follow. The function assumes positive finite areas and allowable stress, and finite nonnegative force. It is a programming model, not a complete structural safety analysis.

1.10. Search combinations without repeating them

Lecture-slide alignment: Week 12 source PDF, pp. 24–25.

The series-resistor exercise uses four ideal integer-ohm catalogue entries: `[2, 4, 8, 10]`. It requests every pair of *distinct entries* whose resistance sums to 12 ohms. The indices represent individual available entries, so one cannot reuse the same entry twice.

```python
# Source companion definition; consult the surrounding notes for its contract.
def resistor_pairs(resistances_ohm, target_ohm):
    """All i<j exact-sum pairs from a catalog of ideal integer-ohm values."""
    matches = []
    for i in range(len(resistances_ohm)):
        for j in range(i + 1, len(resistances_ohm)):
            # Distinct entries; each unordered pair is tested once.
            if resistances_ohm[i] + resistances_ohm[j] == target_ohm:
                matches.append((i, j))
    return matches
```

Code 1.15: Source definition: enumerate exactly the pairs with $i < j$.

```python
from week12_algorithms import resistor_pairs
# Ideal integer-ohm entries; each index denotes one available component.
resistances_ohm = [2, 4, 8, 10]
pairs = resistor_pairs(resistances_ohm, 12)
print('Index pairs:', pairs)
for i, j in pairs:
    print(f'{resistances_ohm[i]} + {resistances_ohm[j]} = 12 ohm')
print('Candidate pairs:', len(resistances_ohm) * (len(resistances_ohm) - 1) // 2)
```

Code 1.16: Worked result for the four-entry series-resistor exercise.

```
Index pairs: [(0, 3), (1, 2)]
2 + 10 = 12 ohm
4 + 8 = 12 ohm
Candidate pairs: 6
```

For n entries, the first index has $n-1$ later partners, the next has $n-2$, and so on. Therefore the number of candidate evaluations is

$$\binom{n}{2} = (n-1) + (n-2) + \cdots + 1 = \frac{n(n-1)}{2}.$$

For four entries this gives six pairs, of which $(0,3)$ and $(1,2)$ succeed. The condition $i < j$ prevents both self-pairing and duplicate reversed pairs. Distinct entries can have equal resistance values; their distinct identities should not be silently removed.

All matches are requested, so this implementation evaluates all candidate pairs even if it finds a match early. With constant-cost integer comparisons, the time grows quadratically and output storage grows with the number of matching pairs. The ideal integer values justify exact addition and equality here; real resistor tolerances are outside this teaching model.

1.11. A running best must keep searching

Lecture-slide alignment: Week 12 source PDF, p. 26.

The four-record example on source page 26 has a single feasibility condition: factor of safety at least 2.0. Its feasible records appear in order B, C, D, with respective costs 150, 180, and 130 USD. Stopping at B answers "first feasible," not "cheapest feasible."

```python
# Source companion definition; consult the surrounding notes for its contract.
def cheapest_feasible(records, required_fos=2.0):
    """Return lowest-cost feasible record, or None; keep first equal-cost tie.

    Assume records have finite numeric cost_USD and fos fields.
    """
    best = None
    for record in records:
        if record["fos"] >= required_fos:
            # Do not stop at the first feasible record: later cost may be lower.
            if best is None or record["cost_USD"] < best["cost_USD"]:
                best = record
    return best
```

Code 1.17: Source definition: retain the lowest-cost feasible record.

```python
from week12_algorithms import cheapest_feasible
# This four-record dataset is distinct from the five-design ranking case.
records = [
    {'name': 'A', 'mass_kg': 3.2, 'cost_USD': 120, 'fos': 1.7},
    {'name': 'B', 'mass_kg': 3.8, 'cost_USD': 150, 'fos': 2.3},
    {'name': 'C', 'mass_kg': 2.9, 'cost_USD': 180, 'fos': 2.1},
    {'name': 'D', 'mass_kg': 4.4, 'cost_USD': 130, 'fos': 2.6},
]
best = cheapest_feasible(records)
if best is not None:
    print('Best feasible:', best['name'])  # Only FoS is constrained here.
    print('Cost:', best['cost_USD'], 'USD')
    print('FoS:', best['fos'])
```

Code 1.18: Worked driver using the full four-record data recovered from Wang, Section 10.7.

```
Best feasible: D
Cost: 130 USD
FoS: 2.6
```

The condition `best is None or ...` uses short-circuit evaluation. When no best record has yet been selected, Python does not attempt to read a cost from `None`. Later, a record replaces `best` only if its cost is strictly smaller. Equal-cost ties therefore retain the first feasible record at that cost.

Invariant

After each inspected record, `best` is either `None` because no inspected record is feasible, or the first lowest-cost feasible record among the inspected prefix. Completing the scan extends this statement to the whole list.

Do not merge two different design exercises

The four-record data in this example come from the course book's Section 10.7, referenced by the slide. D has mass 4.4 kg, but this example constrains only factor of safety. Chapter 3 uses a different five-record dataset with an explicit mass limit. Applying that later limit here would silently change the problem.

1.12. Debugging: absence reported too early

Lecture-slide alignment: Week 12 source PDF, pp. 27–28.

```python
# INTENTIONAL LOGIC DEFECT: one failed comparison does not prove absence.
def broken_search(values, target):
    for i, value in enumerate(values):
        if value == target:
            return i
        return None  # Wrong indentation: remaining candidates are ignored.

print(broken_search([2, 4, 6], 6))  # Wrong; the promised first index is 2.
```

Code 1.19: Intentional defect: the absence return is inside the loop.

```
None
```

Only index 0 is tested. Its value 2 does not equal 6, so the function immediately returns `None`, incorrectly ignoring the target at index 2. The smallest repair moves `return None` outside the loop, exactly as in `linear_search`. No extra condition or second loop is needed.

An empty list does not expose this particular error because falling through the function already returns `None`. A matching first element also succeeds. The last-element case is therefore essential evidence. Retain empty, first, last, duplicate, and absent cases after repairing the defect.

Lecture 34 review checklist

Name the output contract. State units, order, matching rule, and duplicates. Inspect all candidates the contract requires. Distinguish zero from absence. Preserve raw data. Count the selected comparisons or candidate evaluations. Interpret the answer only under the stated engineering assumptions.

1.13. Lecture 34 exit ticket: worked answers

Lecture-slide alignment: Week 12 source PDF, p. 29.

For `[20, 35, 30, 40]` and a strict threshold of 30, index 1 is returned after two comparisons. `[31]` is a one-element example returning index zero. Requesting all qualifying index–value pairs gives `[(1, 35), (3, 40)]` and requires four tests. Sorting first is unnecessary and changes the meaning of the positions: the first event in acquisition order need not be the lowest qualifying value.

The important transition is from a convenient loop to a justified answer. Every returned result should be explainable from the contract, the inspected candidates, and the meaning of the data order.

2 Bisection Search in Ordered Data

Lecture-slide alignment: Week 12 source PDF, pp. 30–55; Lecture 35.

Learning outcomes

Distinguish a stored-list search from numerical root finding; state the ordering assumptions; implement and trace an exact binary search; explain duplicate and insertion-position contracts; locate an interval in a prepared calibration grid; and include preparation cost and original-index meaning.

2.1. Order supplies a reason to eliminate candidates

Lecture-slide alignment: Week 12 source PDF, pp. 30–33.

The calibration voltages `[0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0]` are in ascending order. If the target is 2.5 V and the middle value is 1.5 V, every value at or to the left of the midpoint is at most 1.5 V. None can equal 2.5 V. The entire tested left portion can therefore be discarded.

This is not a probabilistic guess that the target is more likely to be on the right. It is a logical consequence of the ordering assumption. The method is called **binary search** or **bisection search**: each unsuccessful midpoint test leaves roughly half as many candidate positions.

Aspect	Week 3 numerical bisection	Week 12 list bisection
Unknown	A continuous input satisfying a model equation	A position holding a stored target
Evidence	Continuity and a suitable root bracket	Ascending, consistently comparable values
Success	An accepted numerical approximation	A stored value exactly equals the target
Other stopping route	Tolerance or work-limit report	The candidate interval becomes empty

The common idea is elimination by halving. The contracts differ. A stored-list search does not need a sign change. A real root can exist between sample points without being stored as a list element. Returning the last midpoint after a failed exact search would confuse an investigated candidate with a successful result.

2.2. Preconditions are part of the algorithm

Lecture-slide alignment: Week 12 source PDF, pp. 34–35.

The source implementation assumes an ascending sequence with random access by integer index, consistent equality and ordering, and no changes to the sequence during the search. Numeric examples use finite values in compatible units. These assumptions make the elimination step and its complexity argument valid.

If values have mixed physical units, numerical order may not correspond to the intended order. If missing values or NaN are retained without an explicit policy, comparisons may fail to divide the input into the required ordered regions. For structured data, sorting by cost and searching as though it had been sorted by mass does not establish the needed precondition.

The low-level function does not scan the entire list to check sortedness on every call. It trusts the caller's preparation. An $O(\log n)$ lookup plus an $O(n)$ sortedness check is an $O(n)$ operation overall. Validation can be done once for a prepared, unchanged sequence and reused over many queries.

Function contract

Input: an ascending list of consistently comparable values and a target.
Output: any index holding an exact match, or `None` if absent.
Duplicates: any matching index is permitted in this first version.
Mutation: the supplied sequence remains unchanged.

The word *any* is important. The contract does not promise the first duplicate. A test that requires a particular duplicate position when the contract permits any one of them is too restrictive.

2.3. An inclusive candidate interval

Lecture-slide alignment: Week 12 source PDF, pp. 36–37.

The variables `low` and `high` identify the first and last candidate indices, both included. Initially they are 0 and `len(values) - 1`. For an empty list this gives 0 and −1, already an empty interval.

```python
# Source companion definition; consult the surrounding notes for its contract.
def binary_search(values, target):
    """ANY exact-match index, or None; values must be ascending and ordered."""
    low = 0
    high = len(values) - 1
    # If target exists, at least one match remains inside [low, high].
    while low <= high:
        mid = (low + high) // 2
        value = values[mid]
        if value == target:
            return mid
        if value < target:
            low = mid + 1  # Exclude the inspected midpoint.
        else:
            high = mid - 1
    return None  # The candidate interval is empty.
```

Code 2.1: Source exact binary-search definition with inclusive endpoints.

Invariant

If an exact match exists, at least one matching position lies in the current interval [low, high]. Every non-returning pass preserves this statement and strictly reduces the number of candidates.

The condition `while low <= high` permits a one-element interval to be tested. The expression `(low + high) // 2` selects an integer midpoint; floor division chooses the lower middle position when the interval contains an even number of indices. Reading `values[mid]` does not create a sliced sublist.

If the midpoint equals the target, returning `mid` satisfies the contract immediately. If its value is smaller, every position at or below that midpoint can be excluded, so the new lower endpoint is `mid + 1`. Otherwise the consistently ordered value is greater than the target, and the new upper endpoint is `mid - 1`. When the endpoints cross, no candidate remains.

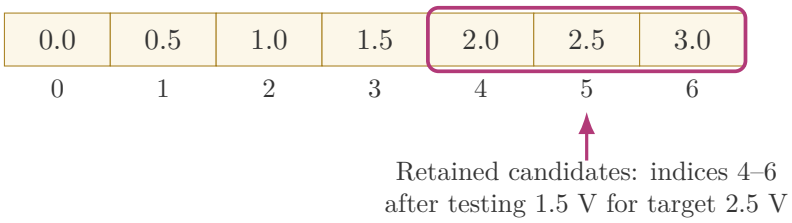


Figure 2.1.: The retained interval is a set of indices, not a new copied list. Based on source page 36.

2.4. Trace a successful and an unsuccessful search

Lecture-slide alignment: Week 12 source PDF, pp. 38–39.

For target 2.5 V in the seven-point grid, the complete source trace is:

Pass	low	high	mid	Value (V)	Next action
1	0	6	3	1.5	Set low to 4
2	4	6	5	2.5	Return 5

The result points to the requested stored value after two midpoint probes. For target 1.7 V:

Pass	low	high	mid	Value (V)	Next action
1	0	6	3	1.5	Set low to 4
2	4	6	5	2.5	Set high to 4
3	4	4	4	2.0	Set high to 3

Now low is 4 and high is 3. The exact result is `None`. The fact that 1.7 lies between stored values is not an exact hit. Interval lookup is a separate task developed later in this chapter.

```
from week12_counts import binary_trace
# An ascending voltage grid in V; rows show bounds before each update.
values = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
for target in [2.5, 1.7]:
    result, rows = binary_trace(values, target)
    print('Target:', target, 'V')
    for row in rows:
        print(row)  # (low, high, mid, value_at_mid)
    print('Exact result:', result)
```

Code 2.2: Reproduce both traces with the source counting companion.

```
Target: 2.5 V
(0, 6, 3, 1.5)
(4, 6, 5, 2.5)
Exact result: 5
Target: 1.7 V
(0, 6, 3, 1.5)
(4, 6, 5, 2.5)
(4, 4, 4, 2.0)
Exact result: None
```

Each printed tuple records the bounds *before* their update. This convention matters when comparing a hand trace with program output. A table that mixes pre-update and post-update bounds can appear inconsistent even when the implementation is correct.

2.5. Progress and the smallest boundary tests

Lecture-slide alignment: Week 12 source PDF, pp. 40–41.

The number of remaining candidates is

$$M = \max(0, \text{high} - \text{low} + 1).$$

On an unsuccessful iteration, at least the tested midpoint is removed. Thus M strictly decreases until the interval is empty. The source warning about `low = mid` can be demonstrated without creating an unbounded loop:

```python
# INTENTIONAL WRONG UPDATE, limited to two passes for safe study.
values = [1, 3]
target = 2
low, high = 0, 1
for step in range(2):
    mid = (low + high) // 2
    print('Bounds and midpoint:', low, high, mid)
    low = mid  # Incorrect: the already-tested candidate remains.
# A while loop using the same update would repeat this interval forever.
```

Code 2.3: Bounded reconstruction of the source wrong-update example.

```
Bounds and midpoint: 0 1 0
Bounds and midpoint: 0 1 0
```

For `values=[1,3]` and target 2, the midpoint is always zero when the bounds remain 0 and 1. Reassigning `low = mid` changes nothing. With the correct `mid + 1`, the first candidate is removed and the process can finish.

The smallest tests often expose the largest conceptual mistakes. `low < high` instead of `low <= high` would skip the final single candidate. Starting `high` at the length rather than length minus one would introduce an invalid candidate index in this inclusive convention.

```python
from week12_algorithms import binary_search
# Source tests plus the two boundary cases requested on the slide.
assert binary_search([], 3) is None
assert binary_search([3], 3) == 0
assert binary_search([3], 4) is None
assert binary_search([1, 3, 5], 5) == 2
assert binary_search([1, 3, 5], 4) is None
assert binary_search([1, 3, 5], 0) is None
assert binary_search([1, 3, 5], 6) is None
print('Seven bisection-search cases passed')
```

Code 2.4: Five source tests plus the two extra boundary cases requested in the slides.

```
Seven bisection-search cases passed
```

2.6. An invalid assumption can produce a quiet wrong answer

Lecture-slide alignment: Week 12 source PDF, p. 42.

```python
from week12_algorithms import binary_search, linear_search
# DELIBERATE PRECONDITION VIOLATION for binary search only.
values = [2, 100, 3, 4, 5]
print('Linear:', linear_search(values, 100))
print('Binary on unsorted input:', binary_search(values, 100))
# The midpoint comparison incorrectly discards the stored target at index 1.
```

Code 2.5: Deliberate violation of the binary-search precondition.

```
Linear: 1
Binary on unsorted input: None
```

The target 100 is present at index 1. Binary search first inspects index 2, containing 3. Since 3 is smaller than 100, it discards the entire left portion, including the target. No Python exception is required for this mistake: the program's logic relies on a false ordering assumption.

Linear search remains correct on this unsorted list because it does not use order to eliminate candidates. The example does not show that binary search is unreliable on valid inputs. It shows why the input contract is part of the algorithm, not merely introductory documentation.

2.7. Duplicates, insertion positions, and exact wrappers

Lecture-slide alignment: Week 12 source PDF, pp. 43–46.

For `[1, 2, 2, 2, 4]` and target 2, the manual binary implementation returns midpoint index 2. The first-match linear function returns 1. Both results satisfy their respective contracts.

```python
from week12_algorithms import binary_search, linear_search, leftmost_match
# Duplicates are permitted, but the returned position depends on the contract.
values = [1, 2, 2, 2, 4]
print('Any binary match:', binary_search(values, 2))
print('First linear match:', linear_search(values, 2))
print('Leftmost exact match:', leftmost_match(values, 2))
```

Code 2.6: Observe the different duplicate policies.

```
Any binary match: 2
First linear match: 1
Leftmost exact match: 1
```

An any-match test should check that the returned index is in bounds and its value equals the target. If the task requires the first equal value, select an implementation that promises that result.

Python's `bisect_left` locates an insertion position before existing equal values. For a consistently ordered ascending list, values before that position are smaller than the target; values from that position onward are at least the target. The result is meaningful even when the target is absent and may equal `len(values)`. This is the insertion-point interface described by the official documentation [5], not an exact-match interface.

```python
from bisect import bisect_left
from week12_algorithms import leftmost_match
# The target 1.7 V is not one of these stored calibration voltages.
values = [0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0]
print('Insertion position for 1.7:', bisect_left(values, 1.7))
print('Exact match for 1.7:', leftmost_match(values, 1.7))
print('Position beyond last:', bisect_left(values, 4.0))
# Position 7 is valid for insertion, but not for values[7].
```

Code 2.7: An insertion position exists even when no exact match exists.

```
Insertion position for 1.7: 4
Exact match for 1.7: None
Position beyond last: 7
```

```python
# Source companion definition; consult the surrounding notes for its contract.
def leftmost_match(values, target):
    """FIRST exact match in an ascending sequence, or None."""
    position = bisect_left(values, target)
    # An insertion position can equal len(values) or point to another value.
    if position < len(values) and values[position] == target:
        return position
    return None
```

Code 2.8: Source wrapper: turn a left insertion position into a leftmost exact match.

The first check, `position < len(values)`, prevents an out-of-range access. Because `and` short-circuits, `values[position]` is read only when the index is valid. The second check confirms equality. Failure of either check returns `None`.

The complementary `bisect_right` gives the position after all equal values. Together they identify the half-open interval containing every exact duplicate:

```
from bisect import bisect_left, bisect_right
# Equal values are contiguous in this sorted sequence.
values = [1, 2, 2, 2, 4]
left = bisect_left(values, 2)
right = bisect_right(values, 2)
print(left, right, values[left:right])  # Stop is excluded when slicing.
print('Number of matches:', right - left)
```

Code 2.9: The equal-key slice and its length.

```
1 4 [2, 2, 2]
Number of matches: 3
```

Here `left=1` and `right=4`, so indices 1, 2, and 3 match. Locating the bounds costs $O(\log n)$; returning a copied slice of k elements additionally costs $O(k)$ time and output storage. Counting matches with `right-left` does not require constructing that slice.

2.8. A calibration interval has its own endpoint policy

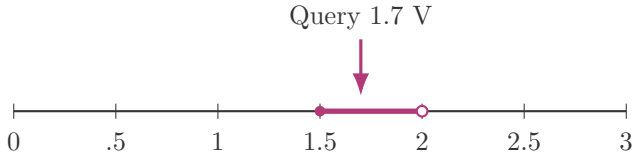
Lecture-slide alignment: Week 12 source PDF, pp. 47–49.

The source calibration table associates voltages 0.0, 0.5, 1.0, 1.5, 2.0, 2.5, and 3.0 V with pressures 0, 50, 100, 150, 200, 250, and 300 kPa. Query 1.7 V lies between endpoint indices 3 and 4, corresponding to 1.5 and 2.0 V. This lookup identifies supporting records; it does not calculate an interpolated pressure.

A value exactly on a grid point touches two neighbouring intervals. The slides deliberately specify ownership:

$$v_i \le x < v_{i+1},$$

except that the final interval also includes the final grid value. Thus 1.5 V belongs to interval 3, on its right, and 3.0 V belongs to interval 5, the last available interval. A value outside $[0, 3]$ returns `None`; no extrapolation is performed.



Interval 3: left endpoint included, right endpoint excluded

Figure 2.2.: The interval convention on source pages 47–49 is explicit, including the separate final-endpoint exception.

```python
# Source companion definition; consult the surrounding notes for its contract.
def prepare_grid(values):
    """Validate once, then freeze a strictly increasing finite numeric grid."""
    if len(values) < 2:
        raise ValueError("a calibration grid needs at least two values")
    prepared = []
    for value in values:
        if isinstance(value, bool) or not isinstance(value, (int, float)):
            raise TypeError("grid values must be real numeric scalars")
        if not isfinite(value):
            raise ValueError("grid values must be finite")
        prepared.append(float(value))
    for i in range(1, len(prepared)):
        if prepared[i] <= prepared[i - 1]:
            raise ValueError("grid must be strictly increasing")
    # A tuple prevents the caller from silently reordering this prepared grid.
    return tuple(prepared)
```

Code 2.10: Recovered source helper: validate once and freeze the numeric grid.

This helper rejects fewer than two points, Boolean or nonnumeric entries, non-finite values, repeated values, and decreasing order. It makes a new list of float values, checks strict increase after conversion, and returns a tuple. Validation and conversion cost $O(n)$ and create $O(n)$ storage. The tuple prevents item assignment or reordering through the returned grid, although the variable name itself can still be rebound to something else.

Scope of the source numeric contract

The helper accepts ordinary finite `int` and `float` inputs representable in the chosen floating-point model; it is not a universal arbitrary-precision conversion service. Extremely large integers can fail conversion in `isfinite` or `float`. The low-level teaching code is retained unchanged rather than silently adding more exception-handling behavior.

```python
# Source companion definition; consult the surrounding notes for its contract.
def calibration_interval(grid, x):
    """Interval i in a prepared grid, or None outside; no extrapolation.

    grid[i] <= x < grid[i+1]; the final interval includes the upper endpoint.
    grid must satisfy prepare_grid's contract. Query validation is constant work.
    """
    if isinstance(x, bool) or not isinstance(x, (int, float)):
        raise TypeError("query must be a numeric voltage")
    if not isfinite(x):
        raise ValueError("query voltage must be finite")
    if x < grid[0] or x > grid[-1]:
        return None
    i = bisect_right(grid, x) - 1
    # An exact last-grid value belongs to the last available interval.
    return min(i, len(grid) - 2)
```

Code 2.11: Source query function for a prepared grid.

The query validation rejects text, Boolean values, NaN, and infinities. It does not repeat grid preparation. Once an out-of-range value is excluded, `bisect_right(grid, x)-1` selects the

interval on the right of an exact interior grid point. At the final point, that expression gives the last point's index; `min(i, len(grid)-2)` maps it to the last *interval* instead.

```python
from week12_algorithms import prepare_grid, calibration_interval
# Validate once; the returned tuple preserves the prepared value ordering.
grid = prepare_grid([0.0, 0.5, 1.0, 1.5, 2.0, 2.5, 3.0])
for voltage_v in [0.0, 1.5, 1.7, 3.0, 3.1]:
    index = calibration_interval(grid, voltage_v)
    if index is None:
        print(f'{voltage_v:.1f} V: outside table')
    else:
        print(f'{voltage_v:.1f} V: interval {index}, '
              f'[{grid[index]:.1f}, {grid[index + 1]:.1f}] V')
# The printed endpoints identify an interval, not an interpolated pressure.
```

Code 2.12: Worked queries exercising interior, endpoint, and outside cases.

```
0.0 V: interval 0, [0.0, 0.5] V
1.5 V: interval 3, [1.5, 2.0] V
1.7 V: interval 3, [1.5, 2.0] V
3.0 V: interval 5, [2.5, 3.0] V
3.1 V: outside table
```

The printed brackets list the two supporting endpoints; the ownership rule remains the half-open convention stated above, with its final exception.

```python
from week12_algorithms import prepare_grid, calibration_interval
# Added tests of the source helper's stated preparation contract.
for values in [[1.0], [0.0, 0.0], [1.0, 0.0], [0.0, float('nan')]]:
    try:
        prepare_grid(values)
    except ValueError:
        print('Grid rejected')
grid = prepare_grid([0.0, 1.0])
try:
    calibration_interval(grid, '0.5')  # Text is not numeric under this contract.
except TypeError:
    print('Text query rejected')
```

Code 2.13: Added checks of documented grid and query rejection.

```
Grid rejected
Grid rejected
Grid rejected
Grid rejected
Text query rejected
```

2.9. Keep the original index with the sorted view

Lecture-slide alignment: Week 12 source PDF, p. 50.

A sorted position and an original position are different coordinate systems. In the raw list `[30, 10, 20]`, value 20 has original index 2 but sorted position 1. The source example carries the original index with each value while constructing an ordered view.

```
from bisect import bisect_left
# Each pair stores (value, original_index); the raw list remains untouched.
raw = [30, 10, 20]
indexed = sorted((value, index) for index, value in enumerate(raw))
keys = [pair[0] for pair in indexed]  # A parallel view built from the SAME pairs.
p = bisect_left(keys, 20)
if p < len(keys) and keys[p] == 20:
    print('Sorted position:', p)
    print('Original index:', indexed[p][1])
print('Raw list:', raw)
```

Code 2.14: Retain a value and its original position as a pair.

```
Sorted position: 1
Original index: 2
Raw list: [30, 10, 20]
```

The expression supplying `sorted` yields pairs `(value, original_index)`. Tuples compare their first component first and then their second on a tie. Thus equal values in this specific view are ordered by original index. The separate `keys` list is built *from that ordered view*; it is not independently sorted and detached from its records. Its position can therefore be used to retrieve the matching pair.

Building the view and keys costs time and storage. Reuse can make that preparation worthwhile, but a single first-event query would not require it. Retain timestamps or identifiers as well when they are the authoritative links back to the raw observations.

2.10. Count what actually happens

Lecture-slide alignment: Week 12 source PDF, p. 51.

The source count table uses an absent target greater than every stored value. The linear counter counts equality comparisons. The binary counter counts *midpoint probes*: one probe can execute both an equality comparison and an ordering comparison. They are separate measures of work and must not be equated with elapsed time or with identical CPU instructions.

```python
from week12_counts import linear_count, binary_count
# Added reproduction of the source count protocol: absent target above all values.
for n in [8, 16, 32, 64]:
    values = list(range(n))
    _, equality_tests = linear_count(values, n)
    _, probes = binary_count(values, n)
    print(n, equality_tests, probes)
# A midpoint probe may include both an equality test and an ordering test.
```

Code 2.15: A small reproduction of the source absent-target counting protocol.

```
8 8 4
16 16 5
32 32 6
64 64 7
```

n	Linear equality tests	Binary midpoint probes
8	8	4
16	16	5
32	32	6
64	64	7
128	128	8
256	256	9
512	512	10
1024	1024	11

For powers of two in this above-the-range task, the count is $\log_2 n + 1$ probes. More generally, the maximum candidate-probe count for this inclusive implementation is $\lfloor \log_2 n \rfloor + 1$ for $n \geq 1$. The logarithmic bound describes candidate elimination with constant-cost indexed access, not a promise about full-program runtime.

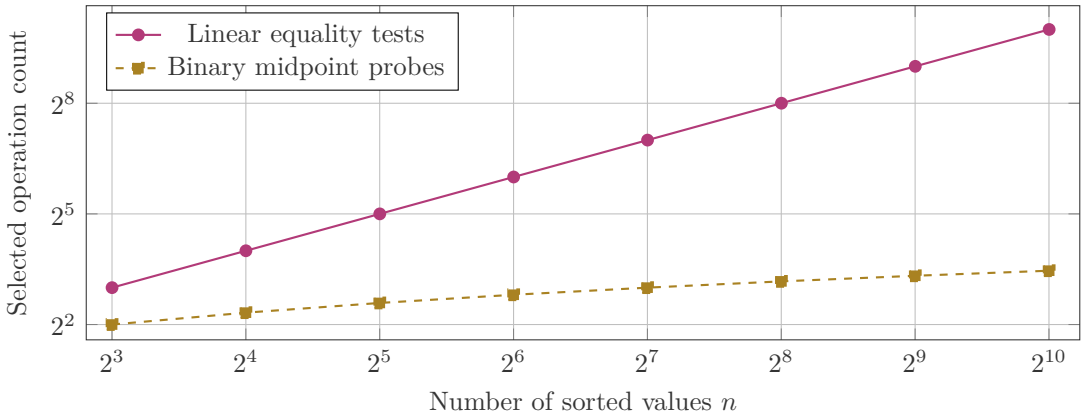


Figure 2.3.: Original source-page-51 counts, redrawn from the retained CSV. Both axes use logarithmic scales. A probe is not necessarily one comparison; neither series is a timing measurement.

2.11. Include preparation and repeated queries

Lecture-slide alignment: Week 12 source PDF, p. 52.

For q exact membership queries on n initially unsorted values, the source comparison is:

Plan	Preparation	Queries
Repeated linear scans	None	$O(qn)$ worst case
Sort once, then bisect	$O(n \log n)$	$O(q \log n)$
Build an ID dictionary	$O(n)$ expected	$O(q)$ expected

An ID dictionary requires a duplicate policy and does not directly locate a neighbouring calibration interval. A sorted copy uses storage and returns positions in a new order. Include maintenance when data change. Expected dictionary costs assume ordinary hashing and bounded-cost keys; they are not worst-case guarantees.

A linear scan can suit one small query. Many queries on unchanged data can justify preparation, but only after stating which result must be preserved.

2.12. Activity, debugging checklist, and exit ticket

Lecture-slide alignment: Week 12 source PDF, pp. 53–55.

```python
from week12_algorithms import binary_search, leftmost_match, lower_bound
# The first two calls seek exact matches; the third seeks a position.
values = [2, 4, 4, 4, 8]
print(binary_search(values, 4))
print(leftmost_match(values, 4))
print(lower_bound(values, 5))  # Index 4 stores 8; 5 is absent.
print(binary_search([], 4), leftmost_match([], 4), lower_bound([], 5))
```

Code 2.16: Worked answer to the three-contract activity.

```
2
1
4
None None 0
```

For `[2,4,4,4,8]`, the manual exact search for 4 returns 2, the leftmost exact search returns 1, and the lower bound for 5 returns 4. The first two identify actual equal elements. The third points to 8, the first value at least 5, because 5 is absent. For an empty list, the exact searches return `None` but a lower-bound search returns insertion position zero. The manual lower-bound body is optional and appears in Chapter 4.

For the exit ticket, `[3,7,9,12]` with target 8 visits midpoint indices 1 and 2. It returns `None`; `bisect_left` returns insertion position 2. Sorting before a single query can cost more than a scan, consume storage, and obscure original positions.

Ordered-search debugging checklist. Check ascending order by the actual searched key. Keep one endpoint convention. Remove a tested candidate on each unsuccessful exact-search pass. Handle absence without invalid indexing. State duplicate and tolerance policies. Tie every returned index to the right ordering of the data.

3 Sorting Algorithms and Engineering Rankings

Lecture-slide alignment: Week 12 source PDF, pp. 56–81; Lecture 36.

Learning outcomes

Specify order, multiplicity, mutation, and ties; use explicit sort keys; trace selection sort and stable merge sort; account for time and extra storage; filter constraints before ranking; and test that the result preserves complete records and raw data.

3.1. Ordered numbers can still describe the wrong record

Lecture-slide alignment: Week 12 source PDF, pp. 56–59.

The source warm-up initially pairs `['steel', 'aluminum', 'copper']` with densities `[7850, 2700, 8960]` in kg/m^3. Sorting only the density list changes which name is associated with which measurement. It makes the numbers ascending but breaks the engineering information.

```
# INTENTIONAL RECORD DEFECT: names and densities initially correspond.
names = ['steel', 'aluminum', 'copper']
density = [7850, 2700, 8960]  # kg/m^3
# Reordering just one column breaks the relationship between fields.
density.sort()
print(names[0], density[0])  # Wrong association: steel 2700.
```

Code 3.1: Intentional defect: reorder a field while leaving its identifiers behind.

```
steel 2700
```

The output is a plausible-looking name and density, yet it assigns aluminum's illustrative density to steel. Store complete tuples, dictionaries, or objects and reorder those records together. A worked tuple-based repair appears in the practice chapter.

Five obligations of a correct sort

The output must have the specified order; contain every input element with the same multiplicity; keep related record fields together; respect the promised input-mutation behavior; and preserve relative order among equal keys when stability is promised.

A result of `[1,3]` for input `[3,1,3]` meets ascending order but loses one occurrence. Checking only neighbouring order is not enough. Sorting is a reordering task, not an implicit request to remove duplicates or discard records.

3.2. Two Python interfaces: new list or changed receiver

Lecture-slide alignment: Week 12 source PDF, p. 60.

The source distinguishes `sorted(raw)` from `raw.sort()`. The first builds a new outer list; the second rearranges the existing list and returns `None`. Both can be appropriate when chosen deliberately. The official Sorting Techniques guide confirms this interface distinction [6].

```
# sorted() creates a new list; list.sort() changes its receiver.
raw = [800, 500, 1200]
ranked = sorted(raw)
print('Raw before sort:', raw)
print('Ranked copy:', ranked)
result = raw.sort()
print('Raw after sort:', raw)
print('Return from sort:', result)
print('Descending:', sorted(raw, reverse=True))
```

Code 3.2: Observe both mutation and return values.

```
Raw before sort: [800, 500, 1200]
Ranked copy: [500, 800, 1200]
Raw after sort: [500, 800, 1200]
Return from sort: None
Descending: [1200, 800, 500]
```

A reference to `raw` held under another variable name would observe the in-place change. By contrast, `ranked` is a different outer list. For a list of dictionaries, however, both outer lists can still refer to the same dictionary objects. “Returns a new list” does not mean “deep-copies all engineering records.”

The keyword argument `reverse=True` reverses the requested comparison ordering. It does not mean “reverse acquisition time” unless time is the chosen sort key. Name the criterion and keep its unit visible in the record and report.

3.3. Keys make a ranking criterion explicit

Lecture-slide alignment: Week 12 source PDF, pp. 61–63.

A key function accepts one input element and returns the quantity used to compare that element with others. A cost key returns `record['cost_USD']`; the entire record remains the element being rearranged. Pass the function itself as `key=by_cost`, not the result of calling `by_cost()` without its required record.

```python
# Added driver using the equal-cost records displayed in the source diagram.
records = [{'name': 'A', 'cost_USD': 100},
           {'name': 'B', 'cost_USD': 100},
           {'name': 'C', 'cost_USD': 90}]
def by_cost(record):
    # Return one comparable key; keep all other fields on the record.
    return record['cost_USD']
print([r['name'] for r in sorted(records, key=by_cost)])
print([r['name'] for r in sorted(records, key=by_cost, reverse=True)])
```

Code 3.3: Worked driver for the equal-cost records on the source stability diagram.

```
['C', 'A', 'B']
['A', 'B', 'C']
```

Python evaluates the supplied key once per input element for the sort. The key should therefore be a clear query, not an operation that unexpectedly changes the record. Lower cost is a ranking rule; it does not establish that a record is feasible or that the cheapest design is best in every engineering sense.

Definition

A **stable sort** preserves the input order of records whose sort keys compare equal. For costs A:100, B:100, C:90, ascending stable sorting produces C, A, B. A remains before B. Descending stable sorting produces A, B, C, still preserving A before B.

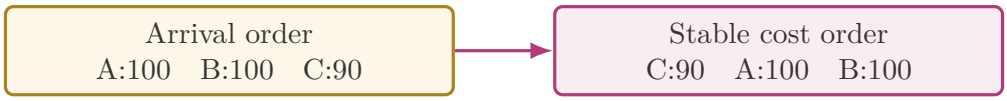


Figure 3.1.: The relative order of A and B survives because their cost keys tie. Redrawn from source page 62.

A tuple key expresses more than one priority. The first component is primary; the second is examined only on a tie in the first, and so on.

```python
# Source companion definition; consult the surrounding notes for its contract.
def design_key(record):
    """Lower cost first; use lower mass to resolve cost ties."""
    return record["cost_USD"], record["mass_kg"]
```

Code 3.4: Source key: cost first, then mass.

```python
from week12_algorithms import design_key
# The tuple key compares mass only when costs tie.
records = [{'name': 'A', 'cost_USD': 100, 'mass_kg': 4.0},
           {'name': 'B', 'cost_USD': 100, 'mass_kg': 3.0},
           {'name': 'C', 'cost_USD': 90, 'mass_kg': 5.0}]
print([r['name'] for r in sorted(records, key=design_key)])
# A ranking alone makes no claim that these records satisfy constraints.
```

Code 3.5: Worked output for the source three-design table.

```
['C', 'B', 'A']
```

C precedes the others because its cost is 90 rather than 100 USD. B precedes A because their costs tie and B has the lower mass. This is not an average of the two criteria and does not assign them equal numerical importance. If both key components tie, stable sorting retains the input order. The units of cost and mass do not need to match because they are compared as separate ordered components, not added together.

3.4. Selection sort fixes one final position per pass

Lecture-slide alignment: Week 12 source PDF, pp. 64–65.

Selection sort repeatedly finds the smallest value in the unfinished suffix and swaps it into the next position. On source input `[7,3,5,2]`, the first pass selects 2 at index 3 and swaps it with 7. The list becomes `[2,3,5,7]`. Later passes still inspect the remaining suffix even though it already happens to be ordered.

```python
# Source companion definition; consult the surrounding notes for its contract.
def selection_sort(values):
    """New sorted list; finite, comparable values; input remains unchanged."""
    result = values.copy()  # Copy references; preserve the caller's list.
    for i in range(len(result) - 1):
        smallest = i
        for j in range(i + 1, len(result)):
            if result[j] < result[smallest]:
                smallest = j
        # Extend the final sorted prefix by one smallest remaining value.
        result[i], result[smallest] = result[smallest], result[i]
    return result
```

Code 3.6: Source selection sort: rearrange a copy of the input.

The first line creates a shallow outer copy. The variable `i` selects the next final position. `smallest` initially points to that position, then tracks the smallest value found so far in its remaining suffix. The inner loop begins at `i+1` because the current candidate at `i` is already known. After the scan, one swap places the smallest remaining value at `i`.

Invariant

At the beginning of pass i, the prefix before i holds the i smallest values in their final ascending positions. The unfinished suffix holds every remaining element. Selecting its minimum and swapping preserves all elements while extending the final prefix by one.

No last pass is needed once all but one position are final. Hence the outer bound is `range(len(result)-1)`. For empty and singleton lists there are no selection passes, but the returned value is still a new outer list.

```python
from week12_counts import selection_count
# Trace the exact four-value list printed in the slides.
raw = [7, 3, 5, 2]
result, comparisons, rows = selection_count(raw)
for number, row in enumerate(rows, start=1):
    print('Pass', number, row)
print('Comparisons:', comparisons)
print('Original:', raw)  # The wrapper rearranges a copy, not the raw list.
```

Code 3.7: Trace the source example with the selected-comparison counter.

```
Pass 1 [2, 3, 5, 7]
Pass 2 [2, 3, 5, 7]
Pass 3 [2, 3, 5, 7]
Comparisons: 6
Original: [7, 3, 5, 2]
```

The second and third passes may exchange an element with itself. This does not change the values, but the comparisons leading to that decision still occur. A sort cannot be classified as constant work merely because a particular displayed list looks unchanged on later passes.

3.5. Selection-sort cost and its stability limitation

Lecture-slide alignment: Week 12 source PDF, pp. 66–67.

For $n \geq 1$, the numbers of data comparisons are $n-1, n-2, \ldots, 1$. Therefore

$$C(n) = \sum_{i=0}^{n-2} (n-i-1) = \frac{n(n-1)}{2}.$$

The four-value example performs $3+2+1 = 6$ comparisons. The count does not depend on whether the list was initially ordered, so both best and worst time are $\Theta(n^2)$ for this implementation as n grows.

The in-place rearrangement loop needs only a few index variables. The source *function*, however, deliberately preserves its caller's list by allocating a new one. That wrapper adds $O(n)$ storage. Report the memory convention: excluding a required output gives a different bookkeeping description from counting all extra allocation.

The usual swap-based selection sort is not stable. Consider three labelled records A:2, B:2, C:1, comparing only their numeric keys. The first swap produces C:1, B:2, A:2. A has moved past B even though their keys are equal. Numeric order is correct, but equal-key arrival order is not preserved.

Labels must not become hidden tie-breaking keys

To demonstrate this counterexample in Python, comparisons must use only the displayed numeric key. Sorting tuples `(key, label)` would introduce label ordering as an additional criterion. The optional executable probe in Chapter 4 uses a small comparison class to keep identity visible without making the label part of the comparison.

3.6. Merge sort divides first and combines on return

Lecture-slide alignment: Week 12 source PDF, pp. 68–69.

Merge sort uses the recursive pattern developed in Week 6. Divide a list into two smaller lists, sort each half, and merge the two sorted results. Length zero or one is a base case: such a list already has ascending order.

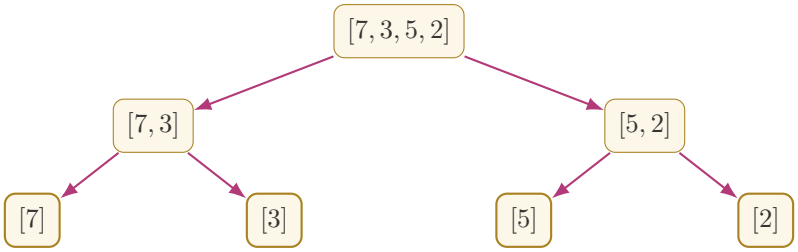


Figure 3.2.: Source-page-68 split structure. The return path merges [7] with [3] into [3, 7], and [5] with [2] into [2, 5], before merging those results.

Two sorted inputs can be merged with two cursor indices. At every step, the smallest unconsumed element of each input is at its cursor. The smaller of those two fronts is the smallest remaining element overall because no hidden smaller value can follow it in its own already-sorted input.

The source merging demonstration separately uses `left=[2,7]` and `right=[3,5]`. Its choices produce `[2]`, `[2,3]`, `[2,3,5]`, then `[2,3,5,7]`. This pair is a standalone illustration of merging; it is not the pair of sorted halves produced by the preceding `[7,3,5,2]` split tree. Both examples have the same final sorted result, but their intermediate steps should not be conflated.

3.7. A stable merge must preserve unconsumed tails

Lecture-slide alignment: Week 12 source PDF, pp. 69–71.

```python
# Source companion definition; consult the surrounding notes for its contract.
def merge(left, right):
    """Merge two ascending lists into a new stable list; preserve inputs."""
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        # Prefer LEFT on a tie, preserving relative order of equal keys.
        if right[j] < left[i]:
            result.append(right[j])
            j += 1
        else:
            result.append(left[i])
            i += 1
    # Exactly one or both tails remain; each tail is already sorted.
    result.extend(left[i:])
    result.extend(right[j:])
    return result
```

Code 3.8: Source stable merge: choose the left element when neither key is smaller.

The variables `i` and `j` locate the unconsumed fronts. The loop continues only while both sides have an available element. If `right[j] < left[i]`, choose the right item; otherwise choose the

left. For a consistent ordering, the latter branch includes a tie, preserving the left element's earlier position across the original split.

Each appended element advances exactly one cursor, so progress is visible. Once either input is exhausted, the remaining tail on the other side is already sorted. Both `extend` statements are necessary: one may append nothing, but the other must retain all remaining elements. Empty inputs are handled by the same structure because the loop is skipped and the tail is copied.

Invariant

The output is sorted and consists of all values already consumed from the left and right inputs, with multiplicity preserved. Every unconsumed value is at least as large as the last appended value under the consistent ordering. On a tie, preserving the left element first retains stable order across the split.

```python
# Source companion definition; consult the surrounding notes for its contract.
def merge_sort(values):
    """New stable sorted list; ordinary comparisons; depth-first recursion."""
    if len(values) <= 1:
        return values.copy()  # Even the base case returns a new outer list.
    middle = len(values) // 2
    left = merge_sort(values[:middle])
    right = merge_sort(values[middle:])
    # Each half is sorted before it is merged with the other half.
    return merge(left, right)
```

Code 3.9: Source recursive merge sort: sort the halves before merging them.

The two recursive calls finish before `merge(left, right)` begins. The base case returns `values.copy()`, not the original list, so even an empty or singleton result satisfies the fresh-outer-list promise. Slicing and merging allocate new lists; no statement rearranges the supplied input.

```python
from week12_algorithms import merge, merge_sort
# Each input to merge must already be sorted.
left = [2, 7]
right = [3, 5]
print('Merged:', merge(left, right))
print('Inputs:', left, right)  # Neither list is changed.
print('Whole sort:', merge_sort([7, 3, 5, 2]))
print('Empty left:', merge([], [1, 2]))  # The entire right tail survives.
```

Code 3.10: A complete merge call and whole-list sorting call.

```
Merged: [2, 3, 5, 7]
Inputs: [2, 7] [3, 5]
Whole sort: [2, 3, 5, 7]
Empty left: [1, 2]
```

3.8. Derive work and peak storage separately

Lecture-slide alignment: Week 12 source PDF, pp. 72–73.

At each recursive level, splitting, slicing, and merging contribute work proportional to the total number of elements on that level. There are logarithmically many levels. For balanced halves, the source recurrence is

$$T(n) = 2T(n/2) + \Theta(n), \qquad T(n) = \Theta(n \log n).$$

For non-power-of-two lengths, the halves have sizes $\lfloor n/2 \rfloor$ and $\lceil n/2 \rceil$; the same growth result applies. This implementation continues dividing and merging already-sorted input, so it does not automatically become linear merely because the values were initially ascending.

A full execution cumulatively copies $\Theta(n \log n)$ elements across its levels. That is not its *peak simultaneously live* storage. The implementation is depth-first: subproblems on unrelated branches are not all active together. Pending halves and produced lists along an active path form geometrically decreasing sizes, while a merge at the largest scale needs linear list storage. The peak extra list storage is therefore $O(n)$, with $O(\log n)$ active call frames. Python object overhead and exact constants are not counted by this classroom model.

Method	Time	Stable?	Input preservation here
Selection sort	$\Theta(n^2)$	No	Makes an outer copy
Insertion sort (optional)	$\Theta(n^2)$ worst	Yes, strict shifts	Makes an outer copy
Merge sort	$\Theta(n \log n)$	Yes, left ties	Returns new lists
Python built-in sorting	$O(n \log n)$ worst	Yes	`sorted`: new list; `sort`: in place

The built-in cost row is the source deck's general-purpose sorting comparison, not a claim that Python runs our small merge-sort function internally. Educational implementations make the mechanics visible. The source recommends documented built-ins for ordinary application sorting.

3.9. Test the entire sorting contract

Lecture-slide alignment: Week 12 source PDF, p. 74.

```python
from week12_algorithms import selection_sort, insertion_sort, merge_sort
# Independent known outputs; duplicates and a fresh output list are required.
cases = [([], []), ([4], [4]), ([3, 1, 3, 2], [1, 2, 3, 3])]
for sorter in [selection_sort, insertion_sort, merge_sort]:
    for raw, expected in cases:
        before = raw.copy()
        result = sorter(raw)
        assert result == expected
        assert raw == before
        assert result is not raw
print('Nine sorter/case checks passed')
```

Code 3.11: Source sorter tests: exact contents, unchanged input, and fresh result.

```
Nine sorter/case checks passed
```

The duplicate-containing expected result checks more than order. The input-copy comparison checks non-mutation. The identity test `result is not raw` checks the fresh-list promise. Equality alone cannot prove that the returned list is a different object.

To test stable sorting, attach visible labels to equal keys and check that their relative order survives. To test complete records, confirm that each identifier remains paired with its own attributes. A test suite should also include already sorted and reversed inputs, multiple equal values, and sizes not divisible by two. Agreement with `sorted` on many examples is useful evidence, but retain independent small expected answers as well.

3.10. Separate constraints from the ranking objective

Lecture-slide alignment: Week 12 source PDF, pp. 75–77.

The source five-design dataset is:

Design	Mass (kg)	Cost (USD)	Stress (MPa)	Strength (MPa)
A	3.5	180	140	250
B	4.2	160	100	250
C	3.8	210	90	250
D	3.9	190	80	200
E	2.9	195	110	300

A record is feasible only when its factor of safety is at least 2, its mass is at most 4 kg, and its cost is at most 200 USD. Among feasible records, lower cost is preferred, with lower mass breaking a cost tie.

The factor of safety is calculated as strength divided by stress, using matching MPa units. Both numerator and denominator therefore cancel to a dimensionless ratio. The source function assumes positive stress and finite numerical fields; it is not a complete data-validation pipeline.

```python
# Source companion definition; consult the surrounding notes for its contract.
def rank_designs(records, required_fos=2.0, max_mass_kg=4.0, max_cost_usd=200):
    """Filter then rank teaching records; preserve caller records and order.

    Assumes finite numeric fields, stress_MPa > 0 and nonnegative costs/masses.
    New dictionaries hold computed FoS; sorted() alone would not copy records.
    """
    feasible = []
    for record in records:
        fos = record["yield_MPa"] / record["stress_MPa"]
        if (fos >= required_fos and record["mass_kg"] <= max_mass_kg
                and record["cost_USD"] <= max_cost_usd):
            enriched = record.copy()  # Do not add fields to the raw record.
            enriched["fos"] = fos
            feasible.append(enriched)
    return sorted(feasible, key=design_key)
```

Code 3.12: Source integration: copy feasible records before adding factor of safety.

For each record, the loop calculates factor of safety and evaluates all stated constraints. On success, it makes a new dictionary, adds the computed `fos` field to that dictionary, and appends it. The final sort uses the previously defined cost–mass key. The copied dictionaries protect raw fields from this enrichment; `sorted` alone would only create a new outer list of references.

Design	FoS	Decision under the stated constraints
A	$250/140 \approx 1.786$	Fails the strength margin
B	$250/100 = 2.5$	Exceeds the mass limit
C	$250/90 \approx 2.778$	Exceeds the cost limit
D	$200/80 = 2.5$	Feasible
E	$300/110 \approx 2.727$	Feasible

```python
from week12_algorithms import DESIGNS, rank_designs
# Source limits: FoS >= 2, mass <= 4 kg, cost <= 200 USD.
ranked = rank_designs(DESIGNS)
for record in ranked:
    print(f"{record['name']}: ${record['cost_USD']}, "
          f"{record['mass_kg']:.1f} kg, FoS {record['fos']:.2f}")
if ranked:
    print('Recommended under stated rule:', ranked[0]['name'])
else:
    print('No feasible listed design')
print('Raw records still lack a fos field:', all('fos' not in r for r in DESIGNS))
```

Code 3.13: The complete report with unchanged raw records.

```
D: $190, 3.9 kg, FoS 2.50
E: $195, 2.9 kg, FoS 2.73
Recommended under stated rule: D
Raw records still lack a fos field: True
```

D is cheaper than E among these two feasible alternatives. E is lighter, but cost has been declared the primary ranking criterion. Changing that priority can change the preferred design without exposing an error in the original code. Fatigue life, availability, environmental impact, uncertainty, and other considerations are not represented by the stated filter and key.

Engineering check

Distinguish a constraint from an objective. A cheap design that fails a required constraint should not win merely because it appears first in a cost sort. Conversely, being feasible does not mean being first under the chosen objective. The recommendation is conditional on the listed candidates, simplified model, input values, and ranking rule.

3.11. One best record does not require a full ranking

Lecture-slide alignment: Week 12 source PDF, p. 78.

If the output contract requests only the best feasible record, a running minimum or `min(..., key=...)` can inspect k feasible records in $O(k)$ work. A complete comparison-based ranking typically requires $O(k \log k)$ work. Choose the result first, then the method.

```python
from week12_algorithms import design_key
# Already feasible teaching records; no full ranking is requested.
feasible = [{'name': 'D', 'cost_USD': 190, 'mass_kg': 3.9},
            {'name': 'E', 'cost_USD': 195, 'mass_kg': 2.9}]
if feasible:
    best = min(feasible, key=design_key)  # One scan selects one best record.
else:
    best = None
print(best['name'] if best is not None else 'No feasible design')
```

Code 3.14: Select one already feasible record without sorting the full collection.

```
D
```

The guard prevents `min` from being called on an empty list without an explicit absence policy. The returned dictionary is an existing record reference, not an independent copy. Selecting one record does not justify deleting or modifying the unselected records in the raw dataset.

3.12. A sorted-looking result may have lost information

Lecture-slide alignment: Week 12 source PDF, p. 79.

```python
# INTENTIONAL CONTRACT DEFECT: this is deduplication, not a valid general sort.
def broken_sort(values):
    result = []
    for value in sorted(values):
        if value not in result:  # Wrong for a multiplicity-preserving contract.
            result.append(value)
    return result
print(broken_sort([1, 1]))  # Expected sorting result: [1, 1].
```

Code 3.15: Intentional defect: remove duplicates while supposedly sorting.

```
[1]
```

The smallest useful failing case contains two equal elements, such as `[1,1]`. A valid multiplicity-preserving sort must return both occurrences, but this function retains only one. The simplest repair for the stated sorting task is to return `sorted(values)` without the uniqueness filter.

This is not merely a cosmetic error. Two identical measurements can be two distinct observations. Two catalogue entries can represent two separate available components. Removing one changes counts and potentially changes later engineering decisions. Deduplication is valid only under its own explicit contract.

3.13. Assignment preparation and worked exit ticket

Lecture-slide alignment: Week 12 source PDF, pp. 80–81.

The source lists Assignment 7 preparation, not a complete assessment brief. Students should be ready to explain their search contract, sortedness or candidate-space assumptions, sorting key, duplicate behavior, mutation choices, tests, and measured or derived work. Moodle supplies the authoritative task, permitted tools, collaboration rules, deadline, and submission format. No new assessed requirement is added here.

For the exit ticket, an ordered result can still be wrong if elements were added or lost, metadata detached, input modified against its contract, or stable ties reordered. On `[4,1,3,1]`, the first selection pass finds the first minimum at index 1 because the comparison is strict. Swapping indices 0 and 1 produces `[1,4,3,1]`. Later equal minima do not replace the selected index during that pass.

A merge must append the unconsumed tail because those elements are part of the required output, even after one side is exhausted. Choosing the left input on equal keys makes this merge stable. For one cheapest feasible record, use a running minimum or `min`; for a full ranking, use `sorted` with the declared key.

Lecture 36 in one sentence

Choose a sorting or ranking operation whose order, multiplicity, tie behavior, and mutation contract preserve the engineering meaning of the complete records.

4 Optional Extensions and Comparison Evidence

Lecture-slide alignment: Week 12 source PDF, pp. 82–89.

These sections preserve the source deck's optional status. They offer alternative implementations and stronger explanations for students who have already understood the core search and sorting contracts. The manual lower-bound example also explains the function used in the Lecture 35 activity; using that function does not require memorizing a second endpoint convention immediately.

4.1. Insertion sort builds a sorted prefix

Lecture-slide alignment: Week 12 source PDF, pp. 82–83.

Selection sort fixes a globally final position on each pass. Insertion sort uses a different invariant: the prefix already processed is sorted, but later elements can still be inserted before elements in that prefix. Hold the current item, shift larger preceding items right, and place the held item in the gap.

```
# Source companion definition; consult the surrounding notes for its contract.
def insertion_sort(values):
    """New stable sorted list; shift only STRICTLY larger preceding values."""
    result = values.copy()
    for i in range(1, len(result)):
        current = result[i]  # Hold it while making room in the sorted prefix.
        j = i - 1
        while j >= 0 and current < result[j]:
            result[j + 1] = result[j]
            j -= 1
        result[j + 1] = current
    return result
```

Code 4.1: Source optional insertion-sort implementation.

The variable `current` preserves the item while assignments temporarily overwrite its old position. The cursor `j` starts just before that item and moves left. The condition `j >= 0 and current < result[j]` tests the boundary first. Short-circuit evaluation prevents access through an unintended negative index after the scan moves past the beginning.

Only *strictly larger* preceding elements shift. Equal elements do not move past the newly inserted element, preserving their original relative order. A change from a strict to a non-strict shift criterion would require reconsidering the stability promise.

```python
from week12_algorithms import insertion_sort
# The same function handles already ordered, reversed, and duplicate data.
for values in [[1, 2, 3], [3, 2, 1], [4, 1, 3, 1]]:
    print(values, '->', insertion_sort(values))
# Inspect the source while condition: only strictly larger values shift.
```

Code 4.2: Worked examples with ordered, reversed, and repeated values.

```
[1, 2, 3] -> [1, 2, 3]
[3, 2, 1] -> [1, 2, 3]
[4, 1, 3, 1] -> [1, 1, 3, 4]
```

For already ascending input, no element shifts and the outer loop still makes $n-1$ passes, giving $\Theta(n)$ time. For strictly descending input, item i shifts past i preceding elements, so the total shifts are $1+2+\cdots+(n-1)=n(n-1)/2$. The worst time is $\Theta(n^2)$. Data comparisons and boundary-condition checks are not the same selected operation and should be counted separately.

Why nearly sorted input can be cheaper

An inversion is a pair of positions $i<j$ whose values are in the opposite of ascending order. In this strict-shift insertion implementation, each shift removes one such out-of-order relationship. Fewer inversions mean fewer shifts, but a scan of the prefix boundaries and input still costs work. This is an explanation of the source's "few out-of-order pairs" row, not an elapsed-time prediction.

4.2. A manual lower-bound search uses a different invariant

Lecture-slide alignment: Week 12 source PDF, p. 84.

A lower bound is the first position whose value is at least the target; if all values are smaller, the result is the length. This is a position in the range $0,\ldots,n$, so the final position n is legitimate even though `values[n]` is not a valid element access.

```python
# Source companion definition; consult the surrounding notes for its contract.
def lower_bound(values, target):
    """First position with value >= target; len(values) if all are smaller."""
    low, high = 0, len(values)
    # Search positions using a half-open candidate interval [low, high).
    while low < high:
        mid = (low + high) // 2
        if values[mid] < target:
            low = mid + 1
        else:
            high = mid  # This midpoint may still be the first suitable one.
    return low
```

Code 4.3: Source optional half-open lower-bound implementation.

Here `high=len(values)` is an exclusive endpoint and the loop condition is `low < high`. If the midpoint value is smaller than the target, it cannot be the lower bound, so it is discarded with `low=mid+1`. Otherwise it may be the first suitable position, so the algorithm retains it by setting `high=mid`. Termination at `low==high` identifies the insertion boundary.

Invariant

Throughout the loop, every index before `low` has a value smaller than the target, and every existing index at or after `high` has a value at least the target. The unresolved element interval is half-open. When it is empty, the boundary is exactly `low`.

```
from week12_algorithms import lower_bound
# Positions may point to a larger value or lie just beyond the final element.
values = [2, 4, 4, 4, 8]
for target in [1, 4, 5, 9]:
    print(target, '->', lower_bound(values, target))
print('Empty ->', lower_bound([], 5))  # Position 0, not an exact match.
```

Code 4.4: Worked positions, including an empty input and a position beyond the last value.

```
1 -> 0
4 -> 1
5 -> 4
9 -> 5
Empty -> 0
```

Do not copy `high=mid` into the earlier inclusive exact-search loop without changing its invariant and stopping condition. It is appropriate here because the midpoint may be the desired boundary, whereas in the exact-search greater-than branch that midpoint has already failed equality and is not a candidate match.

4.3. Sort complete objects and inspect stability

Lecture-slide alignment: Week 12 source PDF, p. 85; extension of p. 67.

A named key can retrieve an attribute from an object just as it retrieves a field from a dictionary. The element being reordered remains the complete object, including its identifier and other metadata.

```
# A simple object supplies the fields used by the named sorting key.
class Reading:
    def __init__(self, name, pressure_kpa):
        self.name = name
        self.pressure_kpa = pressure_kpa
def pressure_key(reading):
    # The key is a numeric pressure in a common unit.
    return reading.pressure_kpa
readings = [Reading('P-01', 120), Reading('P-02', 90)]
for reading in sorted(readings, key=pressure_key):
    print(reading.name, reading.pressure_kpa, 'kPa')
```

Code 4.5: Source optional object-sorting example with explanatory comments.

```
P-02 90 kPa
P-01 120 kPa
```

The sort does not copy the two sensor objects. It creates an outer list with references in pressure order. A client that edits one referenced object's attributes changes that object wherever it is referenced.

The following optional driver turns the source stability counterexample into an executable test. Its comparison method uses only the numeric key, so labels remain observational evidence rather than extra sort priorities.

```
from week12_algorithms import selection_sort, merge_sort
# Added executable form of the source A:2, B:2, C:1 diagram.
class KeyedRecord:
    def __init__(self, name, key):
        self.name = name
        self.key = key
    def __lt__(self, other):
        # Compare only the numeric key, NOT the identifying label.
        return self.key < other.key
items = [KeyedRecord('A', 2), KeyedRecord('B', 2), KeyedRecord('C', 1)]
print('Selection:', [x.name for x in selection_sort(items)])
print('Merge:', [x.name for x in merge_sort(items)])
```

Code 4.6: Added instrumentation for the source equal-key stability diagram.

```
Selection: ['C', 'B', 'A']
Merge: ['C', 'A', 'B']
```

A new comparison method is not required to use ordinary `sorted(..., key=...)`. It appears only because the educational selection and merge functions compare their elements directly. The printed selection result reverses A and B; the stable merge result retains their original order.

4.4. Original comparison counts are not clock timings

Lecture-slide alignment: Week 12 source PDF, p. 86.

The source deck's optional sorting plot reports these selected data-comparison counts:

n	Selection comparisons	Stable merge comparisons
8	28	15
16	120	43
32	496	115
64	2016	302
128	8128	727
256	32640	1733

Every selection value follows $n(n-1)/2$ independently of input order. Merge comparisons depend on how long both fronts remain nonempty at each merge. The exact count is not always $n \log_2 n$: tail copying remains real work even after comparisons between the fronts stop.

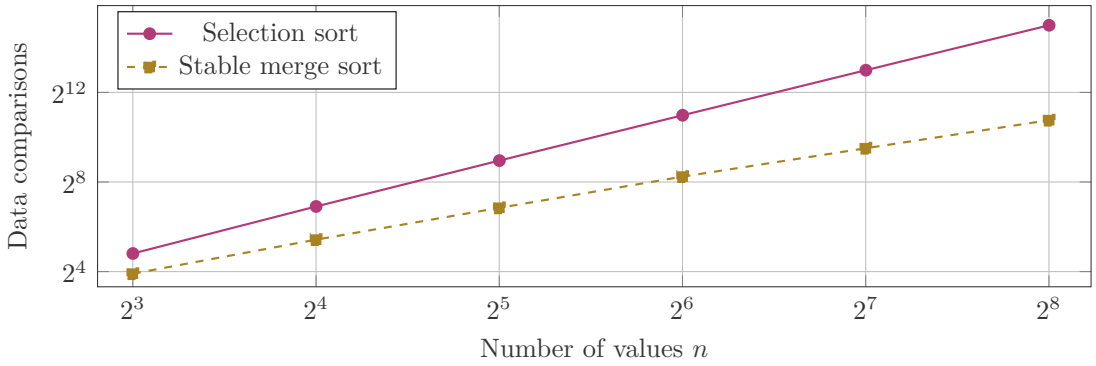


Figure 4.1.: Original source-page-86 values, redrawn from the retained count file. These are reported comparison counts, not elapsed times. The original shuffle metadata are unavailable.

Provenance of the retained plot

Source page 86 says the two algorithms were run on the same seeded shuffled list at each size. The retained CSV records the displayed counts, but the available Week 12 files do not preserve those permutations or their seed. Consequently, the table is retained as reported source evidence. The comparison formula and bounds can be checked, but exact rerunning of its merge counts from the absent input order is not claimed.

The source modules containing the counters are supplied. The additional tool `tools/reproduce_counts.py` starts a new, explicitly seeded experiment and writes new input orders and results to a separate output directory. It never overwrites the original source tables. Its results need not match the old merge column, because it uses independently declared permutations.

4.5. Choose the method by the required result

Lecture-slide alignment: Week 12 source PDF, pp. 87–90.

Need	Starting method	Critical condition
First event in time order	Linear predicate search	Preserve chronology
Any exact hit in ordered data	Binary search	Ascending consistent keys
First equal key or insertion	`bisect_left`	Check equality for an exact hit
One best feasible record	Filter and minimum	State constraints and ties
Full stable ranking	`sorted` with key	Keep records attached
Small candidate combinations	Brute-force enumeration	Cover each admissible candidate

These choices connect directly to the remaining course. A search or sort has a language-independent contract; correctness arguments and test cases can survive translation even when indexing conventions, mutability, syntax, and library interfaces change. Before translating, explain the result in plain language. Before replacing a learning implementation with a library call, verify that the library's contract really matches the task.

The final source message is the habit to retain: a faster answer is useful only when it answers the right question. State the contract, check assumptions, preserve the record, test boundaries, and explain the result.

5 Cumulative Practice and Worked Solutions

These twelve exercises are added for independent practice. They revisit the source contracts and datasets without creating new assessment requirements. Attempt each question before reading its answer. For programming tasks, write at least one expected result and a non-mutation check before executing the solution.

5.1. P1: write the contract before the loop

For pressures `[95,108,99,115,102]`, specify the return type and absent result for (a) membership of 115, (b) its first exact position, and (c) every value strictly above 100. Which tasks permit an early return?

Answer and explanation

(a) Return a Boolean; absence is `False`. (b) Return an integer or `None`. (c) Return all matching index–value pairs or an empty list. Their results are `True`, 3, and `[(1,108),(3,115),(4,102)]`. Tasks (a) and (b) can stop at their first success. Task (c) must inspect all entries. A failed first test never proves absence in an otherwise unexamined list.

5.2. P2: first close versus nearest

For voltages `[9.91,10.02,10.00]`, target 10.0 V, and tolerance 0.1 V, explain the first-close answer. Then implement a nearest-index function that keeps the first index on an equal-distance tie and returns `None` for an empty input.

```
def nearest_index(values, target):
    """Return the first nearest index, or None; assume finite same-unit values."""
    best = None
    best_error = None
    for i, value in enumerate(values):
        error = abs(value - target)  # Compare absolute deviations, not values.
        if best is None or error < best_error:
            best, best_error = i, error  # Strict improvement retains first ties.
    return best
print(nearest_index([9.91, 10.02, 10.00], 10.0))
print(nearest_index([9, 11], 10))
print(nearest_index([], 10))
```

Code 5.1: Added solution: a nearest-reading contract with explicit first-tie behavior.

```
2
0
None
```

The source `first_close` returns 0 as soon as it finds an acceptable error. The new function scans all entries and returns 2. The test `error < best_error` changes the best only for a strictly smaller error, so `[9,11]` around 10 retains index 0. The initial `best is None` test protects the first comparison against `None`. Finite same-unit values remain a precondition. This is a different function, not a silent change to `first_close`.

5.3. P3: justify a catalogue decision

For the area catalogue in Chapter 1, derive the minimum acceptable area independently. Would `first_feasible_area` still return the smallest listed feasible area if the catalogue were reordered? What would happen if no listed area were large enough?

Answer and explanation

$A \geq F/\sigma_{\text{allow}} = 12000/(80 \times 10^6) = 150 \times 10^{-6}$ m^2. That equals 150 mm^2, the final listed value. The helper returns the *first* feasible area in the supplied order, so smallest-feasible reasoning requires ascending candidates. With no passing candidate, the helper returns `None`; the report should say no listed option passes rather than printing an invented area.

5.4. P4: count a complete combination search

List all six index pairs for four resistor entries. Why should the inner loop begin at `i+1`? If two different entries have equal resistance, may both remain in the catalogue?

Answer and explanation

The pairs are $(0,1), (0,2), (0,3), (1,2), (1,3), (2,3)$. Starting after i avoids self-pairing and reversed duplicates. Equal resistance values do not make distinct physical entries identical. Retain each entry unless the catalogue contract explicitly models an unlimited value set instead of individual available components. For the source values, $(0,3)$ and $(1,2)$ sum to 12 ohms.

5.5. P5: trace unsuccessful exact search

Apply the inclusive `binary_search` to `[3,7,9,12]` with target 8. Record pre-update bounds, midpoints, and the final result. What position does `bisect_left` return?

Answer and explanation

First: low 0, high 3, mid 1, value 7; set low to 2. Second: low 2, high 3, mid 2, value 9; set high to 1. Now the interval is empty and exact search returns `None`. The insertion position is 2, before 9. That position is not evidence that 8 is present.

5.6. P6: duplicate policies and safe indexing

For `[1,2,2,2,4]`, compare `binary_search(values,2)`, `leftmost_match(values,2)`, and the range between `bisect_left(values,2)` and `bisect_right(values,2)`. Explain the necessary checks before indexing an insertion position.

Answer and explanation

The specific binary implementation returns midpoint 2; the leftmost wrapper returns 1. The equal range is $[1, 4)$ and has length 3. An insertion position can equal the length or refer to a larger value when the target is absent. Check that it is in range and that the value equals the target. Put the bounds test first when combining the checks with `and`.

5.7. P7: distinguish a grid point from an interval

Prepare the seven-point voltage grid and predict the interval indices for 0.0, 1.5, 1.7, 3.0, and 3.1 V. Explain why a repeated grid point is rejected and whether the lookup returns pressure.

Answer and explanation

The results are 0, 3, 3, 5, and `None`. Interior intervals own their left endpoint; the final interval also owns the last grid point. Repeated points violate strict increase and produce ambiguous or zero-width intervals. The lookup returns an interval index, not a pressure estimate. Interpolation would be a separate algorithm with its own specification.

5.8. P8: repair a detached record

Explain the output `steel 2700` from the independently sorted density column. Repair the representation and print all materials by ascending density without changing their labels.

```
# Added repair of the slide's independently sorted density column.
materials = [('steel', 7850), ('aluminum', 2700), ('copper', 8960)]
def density_key(record):
    return record[1]  # The label remains in the same tuple as its density.
for name, density_kg_m3 in sorted(materials, key=density_key):
    print(name, density_kg_m3, 'kg/m^3')
```

Code 5.2: Added repair using complete tuple records and a named key.

```
aluminum 2700 kg/m^3
steel 7850 kg/m^3
copper 8960 kg/m^3
```

Each tuple couples its name and density. The key selects only the density for comparison, while the tuple remains the element being reordered. The new outer list retains references to the same immutable tuples. The output is correctly ordered *and* correctly associated.

5.9. P9: count and interpret selection-sort passes

Trace the first pass on `[4,1,3,1]`. How many data comparisons occur over the whole sort? Does a correctly ordered result prove stability?

Answer and explanation

The inner scan replaces the initial minimum 4 with the 1 at index 1. The later 1 is equal, not smaller, so index 1 remains selected. The first swap produces `[1,4,3,1]`. There are $3 + 2 + 1 = 6$ comparisons in total. Stable identity order requires a separate check: a swap can move one equal-key record past another even when the keys are correctly sorted.

5.10. P10: reason about merging and storage

Merge `[2,7]` and `[3,5]`. Explain why an empty right input does not mean that the output is complete. Why is the total amount copied across recursive merge-sort levels different from peak extra storage?

Answer and explanation

The front comparisons append 2, 3, and 5. The right input is then exhausted, but 7 remains on the left and must be appended. Copying counts accumulated over the entire execution count the same-sized collection at many levels; those collections are not all live simultaneously. The source depth-first implementation has $\Theta(n \log n)$ cumulative copying and $O(n)$ peak list storage, plus $O(\log n)$ active frames.

5.11. P11: audit a recommendation under changing priorities

Using the five-design dataset, explain why D and E remain after filtering. Which is first with cost as the primary key? Which would be first if mass were primary? Does a changed recommendation imply that the original program was wrong?

Answer and explanation

A fails FoS, B exceeds the mass limit, and C exceeds the cost limit. D and E satisfy all three constraints. D costs 190 USD versus E's 195 USD, so cost-first ranking selects D. E weighs 2.9 kg versus D's 3.9 kg, so mass-first ranking selects E. Changing priorities changes the contract. Different answers can each be correct for their explicitly different objectives; neither establishes universal design superiority.

5.12. P12: distinguish a reference from a copied record

When `find_material` returns a dictionary, does editing it affect the original list's record? Show an independent edit suitable for the flat material record used here. What further question is needed for nested mutable fields?

```python
from week12_algorithms import find_material
# The search itself does not mutate; the returned dictionary is still shared.
raw = [{'name': 'steel', 'density_kg_m3': 7850}]
found = find_material(raw, 'steel')
independent = found.copy()  # New outer dictionary; numeric values are immutable.
independent['density_kg_m3'] = 7800
print('Raw density:', raw[0]['density_kg_m3'])
print('Copied density:', independent['density_kg_m3'])
print('Search returned original:', found is raw[0])
```

Code 5.3: Added reference-and-copy check based on the material lookup.

```
Raw density: 7850
Copied density: 7800
Search returned original: True
```

The returned dictionary is still the original record, but `independent` is a new dictionary. Replacing its density field leaves the original density unchanged. If values themselves contain mutable lists or dictionaries, an outer copy still shares those nested objects. The needed copying depth must follow the promised independence, not a habit of calling `copy()` without inspecting the data structure.

Cross-lecture answer verification

```python
from bisect import bisect_left
from week12_algorithms import first_above, all_above
from week12_counts import binary_trace, selection_count
# Lecture 34: keep the original order when locating the first event.
print(first_above([20, 35, 30, 40], 30))
print(all_above([20, 35, 30, 40], 30))
# Lecture 35: compare absence with a valid insertion position.
result, rows = binary_trace([3, 7, 9, 12], 8)
print('Midpoint indices:', [row[2] for row in rows])
print('Exact result:', result, 'Insertion:', bisect_left([3, 7, 9, 12], 8))
# Lecture 36: the first equal minimum, at index 1, is selected.
print('First selection pass:', selection_count([4, 1, 3, 1])[2][0])
```

Code 5.4: Executable checks of all three source exit tickets' numerical traces.

```
1
[(1, 35), (3, 40)]
Midpoint indices: [1, 2]
Exact result: None Insertion: 2
First selection pass: [1, 4, 3, 1]
```

These calls complement, rather than replace, the explanations above. A correct printed answer is stronger evidence when you can justify each inspected candidate, every bound update, and the output's relation to the original data.

6 Quick Reference and Study Checklist

6.1. Five output contracts to distinguish

Contract	Success	Important caution
Membership	Boolean `True`	Does not provide a location
First exact match	Integer index	Index 0 is valid; `None` means absent
Any exact ordered match	Any matching index	Duplicates need not return the first index
Lower bound	Position in $0, \ldots, n$	A position is not necessarily an exact hit
Full ranking	All records in declared key order	Preserve repeated elements and record fields

6.2. Compact cost reference

These growth descriptions use bounded-cost comparisons and random-access lists. Preparation, returned output, and auxiliary storage must be stated separately.

Operation	Work	Qualification
First linear match	Best $\Theta(1)$; worst $\Theta(n)$	Nonempty input for best-case wording
All qualifying values	$\Theta(n)$	Stores k matching outputs
Every distinct index pair	$\Theta(n^2)$	$n(n-1)/2$ candidate pairs
Binary exact search	$O(\log n)$ worst	Already ascending input; excludes validation
Bounds plus k copied matches	$O(\log n + k)$	Boundary lookup and output cost both included
Selection sort here	$\Theta(n^2)$	New copy adds $O(n)$ storage
Merge sort here	$\Theta(n \log n)$	$O(n)$ peak lists and $O(\log n)$ active frames
One minimum among k records	$\Theta(k)$	Nonempty input or explicit absent policy

6.3. Glossary

Term	Meaning in these notes
Search contract	The result type, matching rule, absence behavior, ordering, duplicate policy, and mutation promise.
Predicate	A yes-or-no condition applied to a candidate, such as strict threshold exceedance.

Term	Meaning in these notes
Candidate	An element, index, record, or combination still under consideration.
Linear search	Inspect candidates in sequence until the required result is established.
First match	The earliest qualifying candidate in the stated input order, not necessarily the smallest or closest.
Brute-force search	Direct evaluation of a specified finite candidate space.
Feasible	Satisfies every declared constraint of the current model.
Objective	The criterion used to prefer one feasible candidate over another.
Tie rule	A declared way to handle equally ranked or equally close candidates.
Invariant	A condition that holds at a stated point before and after each algorithm step.
Progress measure	A quantity that changes toward termination, such as the number of unresolved candidates.
Binary / bisection search	A search that uses sorted order to discard a midpoint and one side when an exact match fails.
Inclusive interval	Both endpoint indices are candidate elements.
Half-open interval	The first endpoint is included and the second excluded.
Insertion position	A place a new value may occupy while preserving order; it can be just beyond the last existing element.
Lower bound	First position whose value is at least the target, or the length if none is.
Equal range	The half-open index range between left and right insertion positions around all equal keys.
Prepared grid	An already validated, strictly increasing numeric sequence suitable for repeated interval queries.
Interpolation	Estimating a quantity between supporting points; not performed by the Week 12 interval lookup.
Sorting key	The value or tuple of values used to compare complete records.
Multiplicity	How many occurrences of each element are present; sorting must not silently change it.
Stable sort	A sort that retains the relative input order of equal-key records.
Selection sort	Repeatedly select the smallest unfinished value and place it in its final position.
Merge	Combine two already sorted sequences into one sorted output while retaining all elements.
Merge sort	Recursively sort two smaller parts, then merge the sorted results.
Insertion sort	Place each next element into its correct location inside an already sorted prefix.
Shallow copy	A new outer container whose elements can still refer to the same nested objects.
Non-mutation	A promise that a function leaves the supplied input state unchanged.

Term	Meaning in these notes
Midpoint probe	One visit to a binary-search midpoint, potentially involving more than one comparison.
Peak live storage	Maximum storage simultaneously retained during execution, not total allocation accumulated over time.
Regression test	A retained check ensuring that an earlier defect does not reappear.

6.4. Readiness for language transfer and course review

Lecture-slide alignment: Week 12 source PDF, pp. 87–90.

You are ready to move forward when you can state a search contract before choosing an algorithm; hand-trace a successful and an absent binary search; explain why a boundary update preserves the invariant; distinguish an original index from a sorted position; preserve complete records and repeated observations; explain the time and memory of the supplied sorts; and justify a recommendation from explicit constraints and a key.

For each self-check, use a tiny example rather than a memorized slogan. Explain one failure case as well as one success. A translation into another language must preserve those results even if array indexing, mutation behavior, or the library interface changes. Do not translate an insertion-position function into an exact-match function by accident, or change a fresh-list function into an in-place operation without updating the contract.

The habit to carry into later engineering courses

Specify, predict, trace, implement, test, and interpret. Only then optimize or replace the learning implementation with a library tool. Correct output, preserved data, and a defensible engineering meaning remain more important than a shorter program or a smaller asymptotic label by itself.

A Source Basis and Complete Slide Coverage

A.1. Primary basis and source distinctions

The primary source is the 90-page *Week 12: Searching and Sorting Algorithms* deck, file `week12_full.pdf`, previously shared in this conversation. Its lecture sources and the recovered `week12_algorithms.py` and `week12_counts.py` provide the implemented contracts. The latter two modules are retained byte-for-byte, with original copies and hashes recorded in the package. Figures are redrawn as editable LaTeX graphics rather than copied as slide screenshots.

The preceding Week 11 reading-note source supplies the document layout and NU branding assets. No lecture timings or classroom participation requirements are turned into self-study assessment rules. Assignment 7 remains a preparation topic, with Moodle the authority for the actual assessment.

Material	Handling in these notes
Printed source definitions	Preserved; comments and line-by-line explanations added around them.
Imported helper definitions	Recovered from the earlier Week 12 modules and printed where needed.
Driver excerpts and omitted imports	Re-created as runnable files; added setup and reporting are worked completions.
Four-design cheapest case	Full values recovered from Wang, Section 10.7, which source page 26 cites. Kept separate from the five-design ranking case.
Calibration ownership	Retain the slides' explicit right-side interior convention and final-endpoint exception, rather than substituting the book's different inclusive-first interval search.
Merge demonstration versus split tree	Preserve both source examples and explain that they are separate demonstrations with different intermediate halves.
Original CSV counts	Preserve unchanged. Missing shuffle metadata are acknowledged; new experiments use separate files.
Practice and verification	Clearly identified additions, not original slide questions or new assessment instructions.

A.2. Reading references

The deck cites Wang's Chapter 10 and Appendix F, Guttag's searching and sorting sections, and official Python documentation. The notes use the deck's implementations where its teaching adaptations differ from the books. In particular, the ranking function enriches copied dictionaries rather than mutating the raw records.

A.3. Page-by-page alignment

The table uses physical source PDF pages, not footer numbers. Each row identifies a reading destination; a section may expand several consecutive slides. Core definitions, examples, diagrams, activities, exit tickets, scope statements, and optional pages are all represented.

Page	Source topic	Reading destination
1	Overview, course scope, and reading route	About these notes, p. 2
2	Overview, course scope, and reading route	About these notes, p. 2
3	Overview, course scope, and reading route	About these notes, p. 2
4	Lecture 34 goals and search contracts	§1.1, p. 6
5	Lecture 34 goals and search contracts	§1.1, p. 6
6	Lecture 34 goals and search contracts	§1.1, p. 6
7	Lecture 34 goals and search contracts	§1.1, p. 6
8	Lecture 34 goals and search contracts	§1.1, p. 6
9	Exact linear search and trace	§1.2, p. 7
10	Exact linear search and trace	§1.2, p. 7
11	Index zero and regression tests	§1.3, p. 8
12	Index zero and regression tests	§1.3, p. 8
13	Index zero and regression tests	§1.3, p. 8
14	Predicate search, index, and timestamp	§1.4, p. 9
15	Predicate search, index, and timestamp	§1.4, p. 9
16	Acquisition order versus sorted order	§1.5, p. 10
17	All qualifying index–value pairs	§1.6, p. 10
18	Normalized names and record references	§1.7, p. 11
19	Normalized names and record references	§1.7, p. 11
20	First close versus nearest	§1.8, p. 12
21	First close versus nearest	§1.8, p. 12
22	Brute-force search and area catalogue	§1.9, p. 13
23	Brute-force search and area catalogue	§1.9, p. 13
24	Distinct resistor-pair enumeration	§1.10, p. 14
25	Distinct resistor-pair enumeration	§1.10, p. 14
26	Lowest-cost feasible record	§1.11, p. 15
27	Premature absence and search checklist	§1.12, p. 17
28	Premature absence and search checklist	§1.12, p. 17
29	Lecture 34 exit-ticket solution	§1.13, p. 17
30	Lecture 35 goals and two kinds of bisection	§2.1, p. 18
31	Lecture 35 goals and two kinds of bisection	§2.1, p. 18
32	Lecture 35 goals and two kinds of bisection	§2.1, p. 18
33	Lecture 35 goals and two kinds of bisection	§2.1, p. 18
34	Ordering and exact-match contract	§2.2, p. 18
35	Ordering and exact-match contract	§2.2, p. 18
36	Inclusive invariant and implementation	§2.3, p. 19
37	Inclusive invariant and implementation	§2.3, p. 19
38	Successful and unsuccessful traces	§2.4, p. 20
39	Successful and unsuccessful traces	§2.4, p. 20
40	Progress, bound updates, and tiny tests	§2.5, p. 21
41	Progress, bound updates, and tiny tests	§2.5, p. 21
42	Unsorted-input counterexample	§2.6, p. 22
43	Duplicates, insertion positions, and equal range	§2.7, p. 22
44	Duplicates, insertion positions, and equal range	§2.7, p. 22
45	Duplicates, insertion positions, and equal range	§2.7, p. 22

Page	Source topic	Reading destination
46	Duplicates, insertion positions, and equal range	§2.7, p. 22
47	Calibration data, preparation, and boundaries	§2.8, p. 24
48	Calibration data, preparation, and boundaries	§2.8, p. 24
49	Calibration data, preparation, and boundaries	§2.8, p. 24
50	Sorted views retain original indices	§2.9, p. 27
51	Counted search evidence and plot	§2.10, p. 27
52	Preprocessing and repeated queries	§2.11, p. 29
53	Activity, checklist, and exit-ticket solution	§2.12, p. 29
54	Activity, checklist, and exit-ticket solution	§2.12, p. 29
55	Activity, checklist, and exit-ticket solution	§2.12, p. 29
56	Lecture 36 goals and sorting obligations	§3.1, p. 30
57	Lecture 36 goals and sorting obligations	§3.1, p. 30
58	Lecture 36 goals and sorting obligations	§3.1, p. 30
59	Lecture 36 goals and sorting obligations	§3.1, p. 30
60	Returned lists and in-place sorting	§3.2, p. 31
61	Named keys, stability, and tuple priorities	§3.3, p. 31
62	Named keys, stability, and tuple priorities	§3.3, p. 31
63	Named keys, stability, and tuple priorities	§3.3, p. 31
64	Selection-sort trace and implementation	§3.4, p. 33
65	Selection-sort trace and implementation	§3.4, p. 33
66	Quadratic comparisons and unstable swaps	§3.5, p. 34
67	Quadratic comparisons and unstable swaps	§3.5, p. 34
68	Recursive split tree and two-front merging	§3.6, p. 35
69	Recursive split tree and two-front merging	§3.6, p. 35
70	Stable merge and recursive merge sort	§3.7, p. 35
71	Stable merge and recursive merge sort	§3.7, p. 35
72	Time, peak memory, and method comparison	§3.8, p. 37
73	Time, peak memory, and method comparison	§3.8, p. 37
74	Sorting regression tests	§3.9, p. 37
75	Five-design constraints, ranking, and report	§3.10, p. 38
76	Five-design constraints, ranking, and report	§3.10, p. 38
77	Five-design constraints, ranking, and report	§3.10, p. 38
78	One minimum versus full ranking	§3.11, p. 40
79	Duplicate-removing sort defect	§3.12, p. 40
80	Assignment preparation and exit-ticket solution	§3.13, p. 41
81	Assignment preparation and exit-ticket solution	§3.13, p. 41
82	Optional insertion sort and shifts	§4.1, p. 42
83	Optional insertion sort and shifts	§4.1, p. 42
84	Optional manual lower-bound invariant	§4.2, p. 43
85	Optional sorting of complete objects	§4.3, p. 44
86	Original sorting counts and provenance	§4.4, p. 45
87	Method-selection reference	§4.5, p. 46
88	Source basis and adaptation record	§A, p. 55
89	Transfer and closing engineering habit	§4.5, p. 46
90	Transfer and closing engineering habit	§4.5, p. 46

Bibliography

[1] Y. Wang. *Programming for Engineers: Week 12—Searching and Sorting Algorithms*. ENG 101, School of Engineering, Nazarbayev University, Fall 2026. Source PDF: 90 physical pages; Lectures 34–36 and optional reference pages. Previously shared course material.

[2] *Course Specification Report: ENG 101—2L—Programming for Engineers, Fall 2026*. Supplied report, 15 August 2026, Session 12. The later user-provided weekly outline gives the same searching-and-sorting topic sequence.

[3] Y. Wang. *Programming for Engineers with Python*. Supplied working draft v20260815. Chapter 10, especially Sections 10.2–10.19 and 10.30; Appendix F. The four-record dataset is in Section 10.7; the five-record dataset is in Section 10.30.

[4] J. V. Guttag. *Introduction to Computation and Programming Using Python*, second edition. Supplied text, Sections 10.1–10.2. Educational search and merge-sort basis cited by the deck.

[5] Python Software Foundation. *bisect—Array bisection algorithm*. Official documentation: `https://docs.python.org/3/library/bisect.html`. Consulted 15 September 2026 to check insertion-position interfaces and performance qualifications; not a replacement source for the course sequence.

[6] A. Dalke and R. Hettinger. *Sorting Techniques*. Official Python documentation: `https://docs.python.org/3/howto/sorting.html`. Consulted 15 September 2026 for sorting interfaces, key functions, and stability.